Chapter 6

Modified Generative Learners for Robotic Trajectory Modelling

Contents

6.1 Introduction **142**

6.2 Recurrent Neural Network **143**

6.3 Spatio-Temporal Model for Trajectory Generation using CNN-LSTM VAE **146**

6.4 Transformer Neural Network **161**

6.5 Attention-based Trajectory Generator Modules for Improved Performance **166**

6.6 Modified Transformer-VAE model **182**

6.7 Summary **194**

The intricacies of a robot's trajectory are characterized by a profound temporal and inter-joint dependency which is through the harmonious coordination of all motor signals that gives rise to the orchestrated path. The system responds to every movement and delicate change in joint angles, creating a series of ripples that influence each successive action. This sequential impact persists until the robot elegantly traces arcs through space, presenting a mesmerizing exhibition of controlled motion. Variational Autoencoder are a type of neural network that excel in learning the underlying structure of data. In the context of robot motion, VAEs can be trained on a dataset of successful robot trajectories, extracting the latent variables that govern these movements. Two distinct models are proposed to address these challenges. A

spatio-temporal model leverages spatial feature learners and sequential dependencies for task planning, while an attention-based model, utilizing Transformer architecture, captures long-range dependencies across time to enhance trajectory prediction. Both models contribute to improving motion planning and control by enhancing precision, agility and adaptability.

6.1 Introduction

In robot trajectory planning, a technological breakthrough facilitates robots to assimilate knowledge from past experiences, seamlessly adjusting their movements. This breakthrough revolves around a specialized neural network equipped with memory capabilities, enabling the retention of vital information about the robot's journey. The augmented memory not only refines the robot's understanding of its environment but also empowers it to plan trajectories and to control the motor neurons, surpassing previous levels of precision intricately. This advanced memory-driven approach lies at the core of examining the interplay between the state-of-the-art Generative AI and robot trajectory planning, thus heralding a new epoch of intelligent and adaptive robotic motion. This research proposes integrating memory-augmented neural networks with temporal and attention mechanisms to capture environmental and sequential dependencies. Additionally, an attention-based model is used to learn long-range dependencies in a reduced latent space. These approaches aim to significantly enhance the precision, adaptability, and efficiency of robot trajectory planning compared to traditional methods. This chapter presents two neural approaches addressing distinct aspects of this challenge.

The first method employs sequential feature learners to capture environmental structure, complemented by temporal processing through recurrent neural networks to model trajectory dependencies over time. The combined architecture enables more comprehensive motion pattern learning. The second approach develops an attention-based model, motivated by its success in natural language processing for handling long-range dependencies. The proposed implementation extends this capability to trajectory planning while introducing a novel mechanism to learn higher-order atten-

tion vectors in reduced latent dimension space. Key contributions of the proposed work includes:

- The CNN-LSTM-VAE model enhances performance by modelling temporal dependencies in robot trajectory planning.
- The Transformer-VAE module achieves precise modelling of complex motion patterns for robot trajectory planning.
- The modified Transformer-VAE, with an expanded hidden dimension vector, improves long-range dependency modelling and computational efficiency while maintaining a low-dimensional latent vector for effective compression and high-quality output, applicable beyond robotics.
- The modification of existing VAE models to improve latent space learning by introducing a trajectory-specific loss function.

6.2 Recurrent Neural Network

Recurrent Neural Networks introduced in the 1990s, represents a pivotal development in the field of artificial intelligence and machine learning [321, 329]. This class of neural networks are specifically designed for sequential data processing, making it a vital tool in various applications, including natural language processing[330, 331], speech recognition[332] and time series analysis[333]. Unlike feed-forward neural networks, RNNs have concealed states that capture information from previous time steps, allowing them to model temporal dependencies. A sample RNN cell representation is visualised in Fig. 6.1, where the unrailed representation exhibits the capability of the model to utilise the same cell for propagating the hidden state to the next time step for predicting the new data. However, they suffer from vanishing gradient problems, limiting their capability to capture long-range dependencies. More sophisticated RNN variations, such as the GRU [334] and LSTM [335], have been developed to solve these problems by exhibiting improved performance in preserving and utilizing contextual information [336]. In this study, an LSTM-based architecture was chosen for trajectory prediction tasks due to its superior ability to handle long-term dependencies and mitigate vanishing gradient issues.

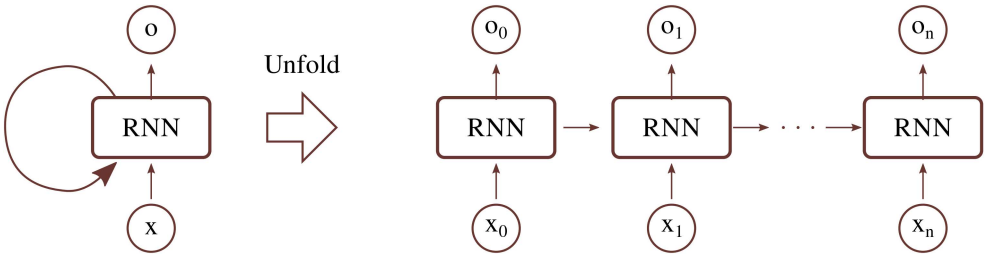


Fig. 6.1: Recurrent cell block diagram with unfolded representation.

6.2.1 Long Short-Term Memory

Long Short-Term Memory is a specialized form of RNN architecture that specialises in processing sequential data due to its distinctive design[335]. LSTMs are endowed with memory cells and gating mechanisms that enable them to capture and maintain information over extended time horizons while mitigating the vanishing and expanding gradient issues often found in traditional RNNs.

LSTM networks have three key components: input, forget and output gates. These gates control information flow through the network. The input gate decides what new information to add, the forget gate removes irrelevant information and the output gate determines what information to pass forward. This structure allows LSTMs to effectively manage and use information over time. Fig. 6.2 represents the LSTM cell with input gate, forget gate and output gate. The mathematical form of

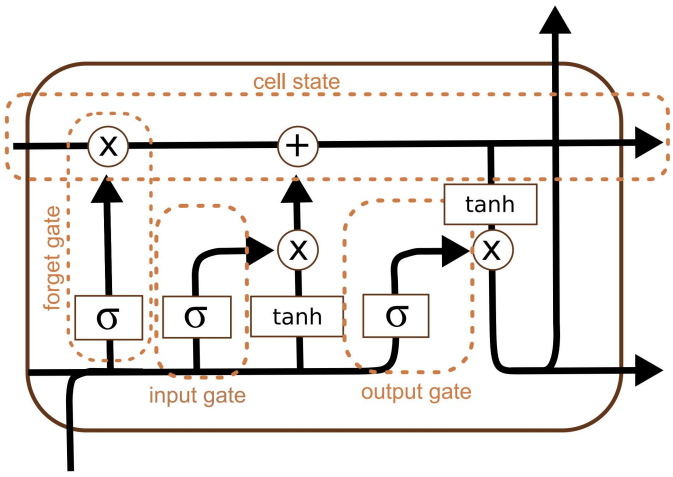


Fig. 6.2: LSTM cell with forget, input and output gates labelled

gating mechanisms forget(f_t), input(i_t) and output(o_t) are represented by Eq. 6.1 - Eq. 6.6 [335].

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad \text{(Forget Gate)} \tag{6.1}$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \text{(Input Gate)} \tag{6.2}$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad \text{(Output Gate)} \tag{6.3}$$

and the cell state and hidden state update are represented in Eq. 6.4 - Eq. 6.6.

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad \text{(Candidate Cell State)} \tag{6.4}$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad \text{(Updated Cell State)} \tag{6.5}$$

$$h_t = o_t \odot \tanh(C_t) \quad \text{(Updated Hidden State)} \tag{6.6}$$

where,

- C_t is the cell state at time t.
- h_t is the hidden state at time t.
- x_t represents the input at time t.
- W_f, W_i, W_o, W_C are weight matrices.
- b_f, b_i, b_o, b_C are bias vectors.
- σ is the sigmoid activation function.
- $\odot$ denotes element-wise (Hadamard) multiplication.

LSTM also offer advantages such as unfolded architectures, reduced model size and differentiable memory similar to RNN. The unfolding characteristic refers to the shared model being reused across each time step, leading to a more compact representation and minimizing redundancy in parameters. This results in a smaller model size, making LSTMs computationally efficient. Additionally, LSTMs possess differentiable memory, allowing them to selectively store and retrieve information over sequential steps, enhancing their ability to capture and retain long-term dependencies in sequential data.

6.3 Spatio-Temporal Model for Trajectory Generation using CNN-LSTM VAE

Robot trajectory data typically shows a solid sequential pattern; including the LSTM networks, which provides a powerful solution. The robot trajectory, denoted as T, represented as $T = \{(q_{1i}, q_{2i}, \ldots, q_{mi})\}$, captures the configuration of the robot at each time step i. The sequential dependency in the robot trajectory is expressed through the ordered set T, where the joint angles at each time step are influenced by the preceding ones. By introducing an LSTM layer into the design, robots may successfully extract and simulate the underlying temporal relationships within the trajectory data. The benefits of LSTM in this application are notable: these networks can capture long-range connections, making them well-suited for jobs that require understanding the context of earlier stages in a trajectory. Additionally, LSTMs are equipped with gating mechanisms that offer fine control over what information to retain and what to discard, thereby boosting their capacity to manage noisy or incomplete trajectory data.

A comprehensive explanation of the VAE architecture is provided in Section 3.6, where the structural and functional aspects of the model are examined in detail. The proposed VAE model is designed to effectively capture the spatial and temporal features in robot sequential data by leveraging convolutional and recurrent layers. The encoder section processes the input trajectory and task label using M convolutional layers, which extract spatial features from the input sequence. The extracted features are then passed through N stacked LSTM layers, enabling the model to capture long-term dependencies in the sequential data. The latent representation, denoted by z, is sampled using the reparametrization trick, ensuring smooth backpropagation through the stochastic layer and allowing the model to function as a generative model.

The decoder section takes z and the task label as inputs, followed by a linear layer and N stacked LSTM layers to recover the temporal dependencies. Finally, M transpose convolution layers reconstruct the output trajectory, ensuring that the generated trajectory has the same dimensions as the input trajectory. The VAE loss function, as defined in Eq. 3.7, comprises two primary terms: the Reconstruction Loss ($\mathcal{L}_R$) and the Kullback-Leibler Divergence Loss ($\mathcal{L}_{KL}$). A single joint space loss as

the reconstruction loss treats all joint errors equally, disregarding the kinematic chain properties, which may lead to suboptimal performance. To address this limitation, the chapter proposes a modified loss function that combines the joint space loss and the robot displacement loss. This formulation considers the kinematic chain, ensuring that errors in individual joints are weighted differently based on their effect on the end-effector position. The proposed reconstruction loss, defined in Eq. 6.7, is expressed as:

$$\mathcal{L}_R = \underbrace{\frac{1}{n}\sum_{i=1}^{n}(q_{0_i} - q_{1_i})^2}_{\text{joint space loss}} + \underbrace{\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{J}(f_j(q_{0_i}) - f_j(q_{1_i}))^2}_{\text{robot distance loss}} \tag{6.7}$$

where,

q_0 - Joint position vector of the original trajectory $q_0 \in \mathbb{R}^J$

q_1 - Joint position vector of the reconstructed trajectory, where $q_1 \in \mathbb{R}^J$.

J - Total number of joints in the robot model.

n - Number of data samples in the batch.

f - Forward kinematics function that maps joint space positions to Cartesian space.

f_j - Forward kinematics output at the jth joint,

and the combined loss of the model as,

$$\mathcal{L}_{VAE} = \frac{1}{n}\sum_{i=1}^{n}(q_{0_i} - q_{1_i})^2 + \frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{J}(f_j(q_{0_i}) - f_j(q_{1_i}))^2 + \beta\mathcal{L}_{KL} \tag{6.8}$$

The loss function implementation requires computing the forward kinematics of the robot, which is computationally expensive; therefore, the module is implemented using the PyTorch Kinematics library [337], leveraging the robot's URDF file and the computations are efficiently carried out on CUDA cores for accelerated performance.

The proposed model is shown in Fig. 6.3. Table 6.2 provides details on the layer descriptions, configurations and trainable parameters of the encoder and decoder models. The number of spatial and temporal feature learner layers, along with kernel size, dropout and the type of recurrent layer (bidirectional or unidirectional), are considered hyperparameters and are detailed in Table 6.1.

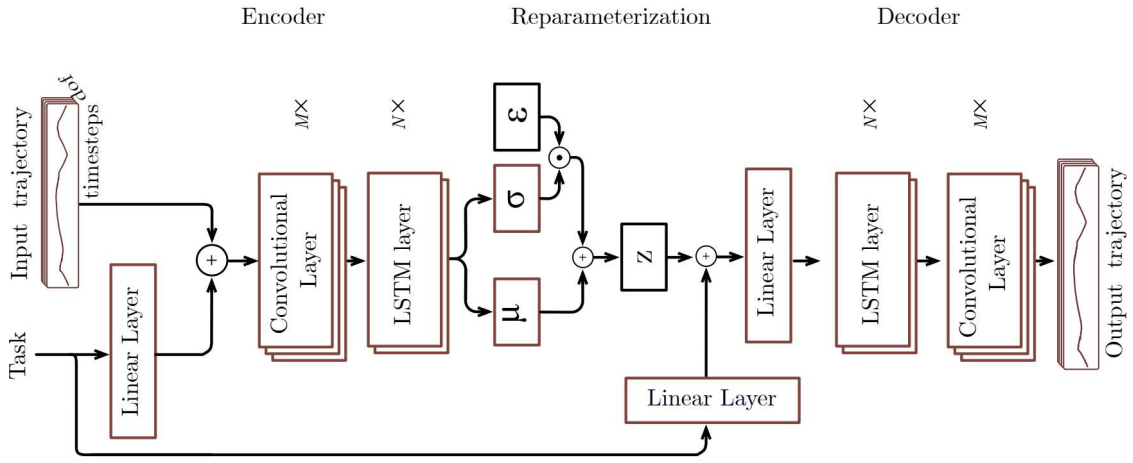


Fig. 6.3: Architecture of a Variational Autoencoder with Recurrent Layers for Robot Sequential Data. The model takes the robot task label and the corresponding trajectory as input to the encoder. The encoder and decoder sections are equipped with M layers of convolutional layers, which extract spatial features. The recurrent layers consist of N stacked LSTM layers, capturing the temporal dependencies in the sequential data. The VAE employs a reparametrization trick to allow for backpropagation through the stochastic layers, enabling it to function as a generative model.

Table 6.1: Hyperparameter under consideration

Hyperparameter	**Search Space**	**Number of combinations**
n_layers (M)	3, 4, 5, 6	4
bidirectional	True, False	2
kernel size	3, 5, 7, 9, 11	5
number of layers rnn (N)	2, 3, 4	3
dropout	0.0, 0.2	2
Total number of combinations		240

Table 6.2: LSTM encoder architecture with latent dimension of 32 and hidden vector dimension of 128

Layer		Configuration	Params
Encoder			
Convolution Layers			
	conv0	Conv1d(17, 32, kernel_size=(11,), stride=(2,), padding=(5,))	6 K
	conv1	Conv1d(32, 64, kernel_size=(11,), stride=(2,), padding=(5,))	23 K
	conv2	Conv1d(64, 128, kernel_size=(11,), stride=(2,), padding=(5,))	90 K
LSTM		LSTM(128, 128, num_layers=2, batch_first=True, dropout=0.2, bidirectional=True)	659 K
mu		Linear(in_features=4096, out_features=32, bias=True)	131 K
logvar		Linear(in_features=4096, out_features=32, bias=True)	131 K
Total trainable parameters			1.04 M
Decoder			
Linear projection		Linear(in_features=42, out_features=2048, bias=True)	88 K
LSTM		LSTM(64, 64, num_layers=2, batch_first=True, dropout=0.2, bidirectional=True)	166 K
Convolution Layers			53 K
	conv0	ConvTranspose1d(128, 32, kernel_size=(11,), stride=(2,), padding=(5,))	45 K
	conv1	ConvTranspose1d(32, 16, kernel_size=(11,), stride=(2,), padding=(5,))	6 K
	conv2	ConvTranspose1d(16, 8, kernel_size=(11,), stride=(2,), padding=(5,))	1416
	conv3	Conv1d(8, 7, kernel_size=(11,), stride=(2,), padding=(5,))	623
Total trainable parameters			306 K

6.3.1 Hyper-parameter optimisation

Hyper-parameter optimisation is a critical aspect of training deep models with large number of parameters. Model training is a computationally costlier process, as it requires much more resources and time. To optimise hyperparameters effectively, techniques like grid search, random search and Bayesian optimisation can be utilised.

Grid search [338] samples the entire parameter space and run all configurations. The number of executions in grid search is $\prod n_i$, where n_i is the number of possible combination of parameter i. While the random search samples the combinations randomly and generate runs. Fig. 6.4 shows Grid and Random sampling of two parameters, in which one is an important parameter and the other is unimportant. The effect of unimportant parameter may not cause a significant difference in the output. So, while considering the figure, the random sampling performs better. In the case of grid search, 3 values per variables are obtained, while in random search, there is a less chance to have same variable twice. So Grid search is not recommended

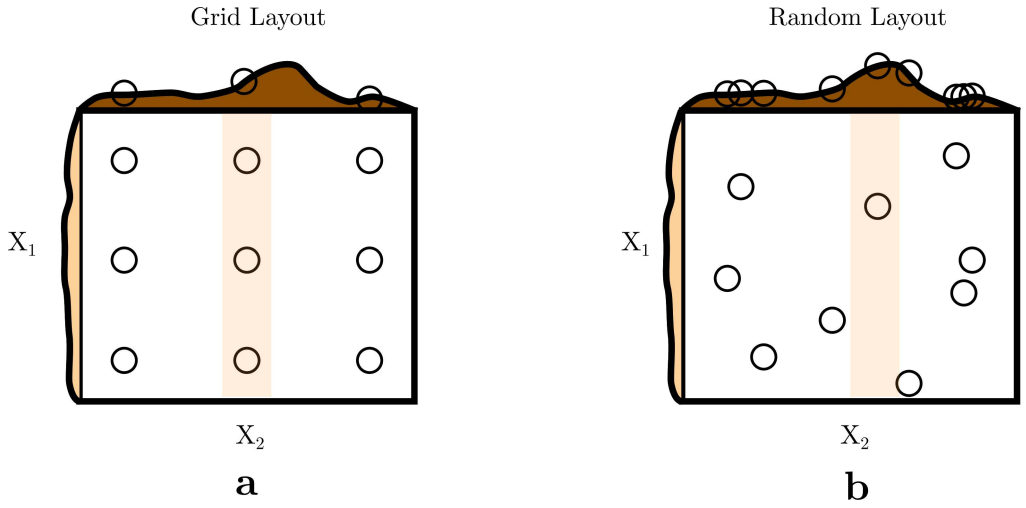


Fig. 6.4: Hyperparameter optimisation techniques with important parameter (X_2) and unimportant parameter (X_1) a) grid search b) Random search.

for larger dimensions, albeit it can find the best runs. On the other hand random search does not guarantee the best parameters, whereas it provides better results faster in high dimensional spaces.

Bayesian optimisation [339] is an advanced technique for hyperparameter optimisation that has proven to be highly effective in which the output value of previous

runs is considered for next sampling. Bayesian optimisation begins by constructing a surrogate model, typically through a Gaussian Process (GP), which models the objective function (e.g., model performance) as a probabilistic distribution. This surrogate model approximates the true performance landscape of the generative model across the hyperparameter space. An acquisition function, often chosen as Expected Improvement (EI) or Probability of Improvement (PI), guides the search by balancing exploration (trying hyperparameters that are uncertain but might lead to improvements) and exploitation (focusing on areas where the surrogate model predicts better performance). Bayesian optimisation starts with an initial set of hyperparameter configurations, which are used to fit the surrogate model. These configurations can be chosen based on prior knowledge or randomly. The optimisation process is iterative. In each step, the acquisition function suggests the next set of hyperparameters to evaluate. The generative model is trained with these hyperparameters and its performance is then recorded. Bayesian optimisation continues until a stopping criterion is met, such as a predefined number of iterations or a target performance level.

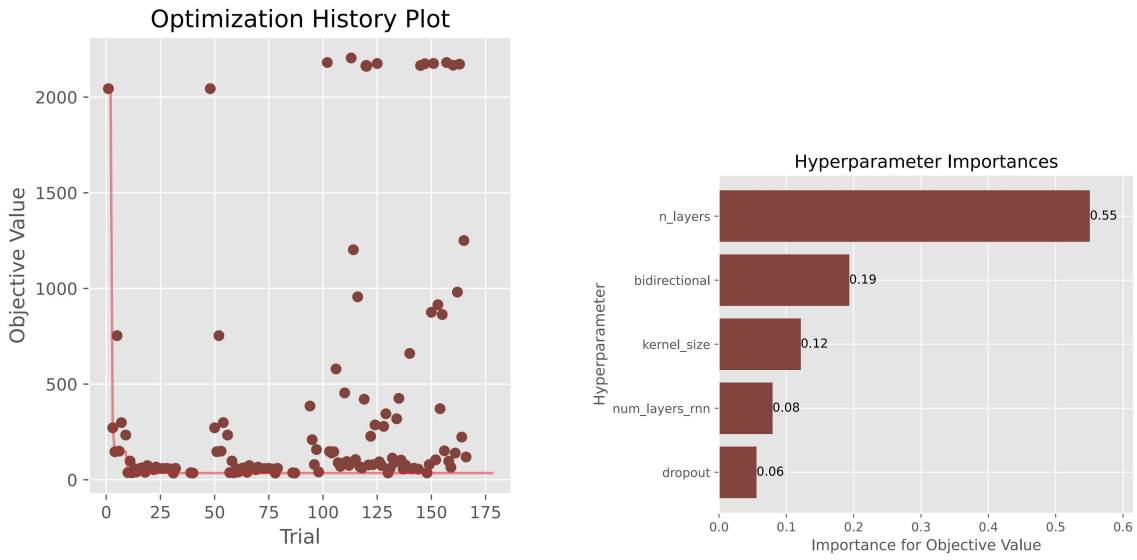
During training, the validation loss is periodically evaluated on a separate validation dataset. This is essential for assessing how well the model generalizes to the unseen data. A critical hyperparameter in early stopping [340] is the patience. Patience determines how many epochs or training iterations the algorithm should wait for the performance metric to improve. If there is no improvement within this threshold, the training is halted. To ensure that the best model is saved, checkpoints are stored each time the validation metric improves. These checkpoints represent the model's state at that point, allowing for the recovery of the best-performing model if early stopping is triggered. Training terminates when the patience limit has been reached. The model associated with the best validation performance is then selected as the final model. This model is expected to generalize well to unseen data. An inherent limitation of early stopping is its exclusive consideration of only the current run. During the execution of hyperparameter tuning, numerous instances are simultaneously being run. This provides an opportunity to terminate the poorest-performing runs by monitoring intermediate performance metrics, such as the validation loss.

Hyperband algorithm [341] is designed for efficient resource allocation in hyperparameter optimisation. Hyperband employs a form of successive halving, where different configurations are trained for different durations, allowing for the allocation of computational resources where they are the most effective. The process begins with a set of hyperparameter configurations or models. These configurations are often randomly sampled from a predefined search space. The configurations/models are trained with the allocated resources. After training, their performance is assessed using a validation metric. This metric could be the validation loss, accuracy or any other relevant measure. After evaluating the configurations, a fraction of them is eliminated. The criterion for elimination could be based on their relative performance. For example, the worst-performing half may be discarded. The remaining configurations/models are given additional resources. This typically involves doubling the resource allocation for the survivors. It is repeated iteratively until only one configuration remains, which is assumed to be the best-performing one. It efficiently allocates resources to promising configurations while discarding underperforming ones, saving time and computational power.

Bayesian Optimisation and Hyperband algorithm (BOHB) [342] are well-suited for parallel execution, making it effective in distributed computing environments. It can evaluate multiple hyperparameter configurations concurrently, further speeding up the optimisation process. BOHB, like Bayesian optimisation, utilizes termination criteria to decide when to stop the optimisation process. These criteria can be based on factors like the number of iterations, computational budget or target performance.

Hyperparameter optimisation is a critical aspect of enhancing model performance, where different parameter configurations are evaluated to minimise validation loss. The process involves iterating through various combinations to identify the best-performing settings. A random forest regressor [343] is employed to assess the importance of these hyperparameters, providing a clear understanding of their influence on the model's predictive capabilities. The analysis highlights that the number of spatial features holds the highest importance, contributing 55% to the model's performance. Bidirectional LSTM architecture follows with 19% significance, while kernel size accounts for 12%. The number of LSTM layers and the dropout rate contribute

8% and 6%, respectively. This hierarchical breakdown effectively underscores the relative significance of each parameter in the optimisation process. Fig. 6.5 visualises the historical progression of hyperparameter optimisation, mapping the relationship between validation loss and trial iterations. It highlights the best-performing configuration derived from the search, as detailed in Table 6.1. Further, Fig. 6.5b presents the hyperparameter importance plot, offering a graphical representation of the relative contribution of each parameter in shaping the model's performance.



(a) (b)

Fig. 6.5: Results of Hyperparameter Optimisation. (a) Optimisation history showing the evolution of the best performing model's objective value across multiple trials for different optimisation algorithms. (b) Relative importance of various hyperparameters, indicating their impact on the model's performance based on the optimisation results.

Following the culmination of hyperparameter optimisation, the optimal model configuration has been identified as *'bidirectional': True, 'dropout': 0.0, 'kernel_size': 9, 'n_layers_cnn': 3, 'num_layers_lstm': 2.* This configuration, noted for its superior performance, has been meticulously selected as the best-performing model. Subsequently, this discerned model configuration is earmarked for subsequent phases of evaluation and utilization in the broader analytical context. The strategic choice of this configuration serves as a cornerstone for the model's robustness and efficacy in

addressing the intended objectives, thereby contributing to the overall success of the optimisation endeavour.

6.3.2 Performance Evaluation

The proposed deep neural network architecture consists of a Variational Autoencoder model, meticulously trained over a duration of 20 epochs. The decision to train the VAE for 20 epochs is based on hyperparameter optimization, where 20 epochs were found to provide the best balance between reconstruction quality, latent space regularization and computational efficiency. The loss plateaus after 20 epochs, indicating that further training would bring only marginal improvements. The VAE loss is utilized as part of the training objective and optimisation is performed using the ADAM optimiser with a fixed learning rate of $1e^{-4}$, complemented by a learning rate scheduler, specifically ReduceLROnPlateau [344]. A learning rate of 1×10^{-4} is selected due to its balance between convergence speed and stability, ensuring gradual progression towards the optimal solution and reducing the risk of overshooting the minima. The Adam optimiser is chosen because it combines the advantages of both the AdaGrad[345] and RMSProp [346] algorithms, dynamically adjusting learning rates based on first-order and second-order moment estimations to handle sparse gradients and non-stationary objectives effectively. Furthermore, a mini-batch size of 32 has been chosen considering the available GPU memory, ensuring a balance between computational efficiency and gradient stability.

The analysis of loss trends over multiple epochs is crucial to assess the model's capacity to generalize and adapt to unseen data. A well-trained model demonstrates convergence of both training and validation losses without any significant divergence, which is indicative of minimal overfitting. Over the course of 20 epochs, the observed trajectories of training and validation losses reveal consistent behaviour, confirming the absence of overfitting. This implies that the model maintains a balance between learning from the training data and generalizing effectively to new scenarios. Fig. 6.6 illustrates the loss versus epoch for both the training and validation sets, providing a clear representation of the model's performance over the training period.

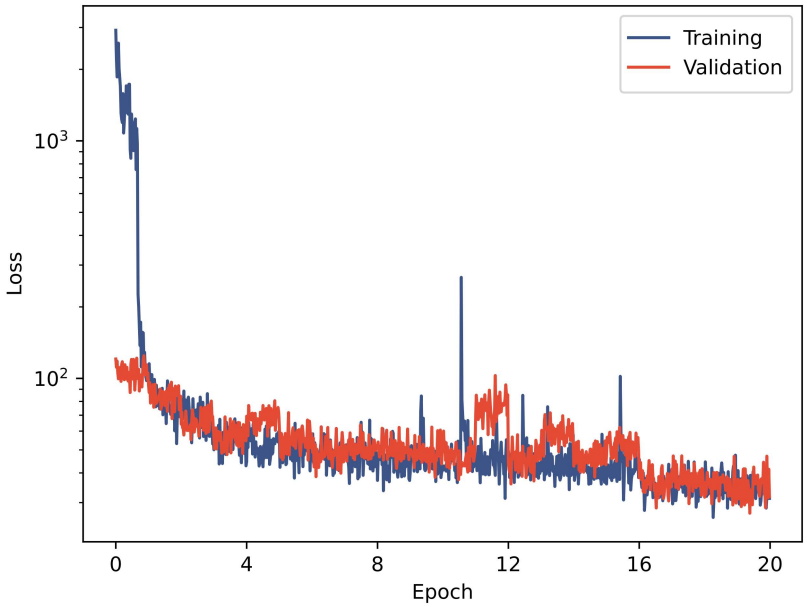


Fig. 6.6: Evolution of VAE loss of the best performing CNN-LSTM-VAE model.

Feature map visualization of hidden layers in both CNN and LSTM components provides a deeper understanding of the learned patterns involved in modelling robot trajectories. The filter maps from each layer reveal the transformation of the embedded latent vector into a fully articulated motor neuron trajectory representation. This transformation highlights the hierarchical process by which abstract features evolve into concrete trajectory representations. Understanding these patterns is essential for assessing the model's capacity to capture and represent complex temporal dependencies inherent in robot trajectory data. The scrutiny of these feature maps offers valuable insights into the internal dynamics of the model, enhancing the interpretability and reliability of trajectory predictions. Fig. 6.7 presents this feature map visualization, demonstrating the model's latent learning capabilities and their impact on trajectory fidelity. Fig. 6.7a–6.7c show the first 25 filters of the convolutional layers, while Fig. 6.7d illustrates the model's output, highlighting how the model extracts and encodes features across layers.

Accurate reconstruction of robot trajectories is essential for evaluating the model's ability to capture and reproduce the intricate temporal dependencies inherent in movement patterns. A well-trained model preserves the salient features and

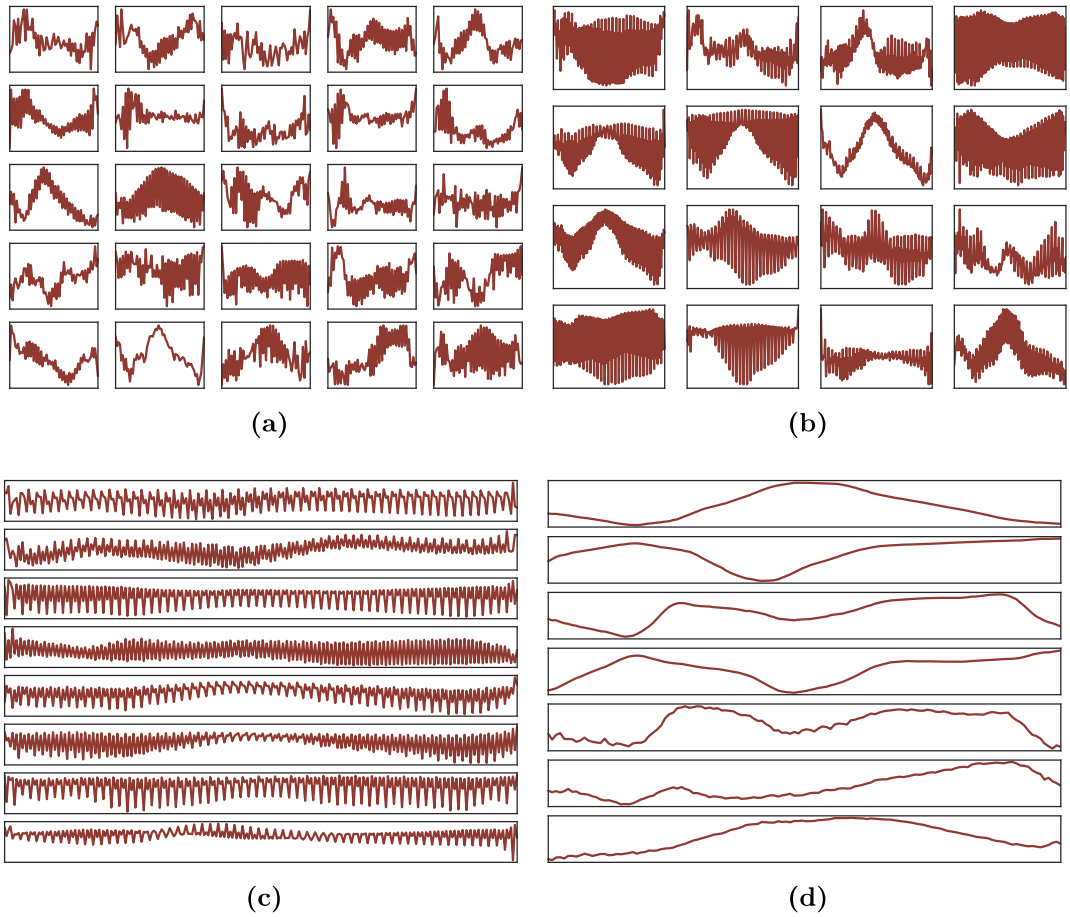


Fig. 6.7: Learned feature maps of generator model from the category *Task1* in the dataset. (a–c) First 25 filters from successive convolutional layers; (d) Model output highlighting the encoded feature representations.

characteristics of the original trajectory, ensuring that the reconstructed trajectory closely aligns with the input. This fidelity in reconstruction highlights the model's effectiveness in learning and generalizing movement patterns, which is critical in tasks involving robot trajectory modelling. The close adherence of the reconstructed trajectory to the original input further emphasizes the model's proficiency in retaining essential nuances of the robot's movement. Fig. 6.8 illustrates this visual comparison between the input (Fig. 6.8a) and reconstructed trajectories(Fig. 6.8b), demonstrating the model's capacity to faithfully replicate the input trajectory. This alignment of the reconstructed trajectory with the original input not only validates the model's learning capabilities but also instils confidence in its applicability for tasks demanding accurate and faithful representation of robot trajectories.

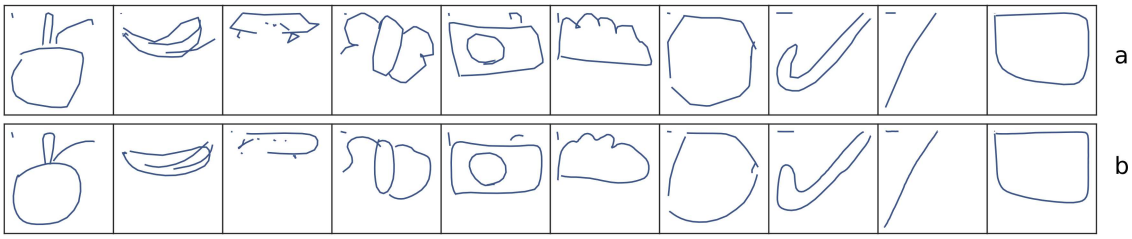


Fig. 6.8: A visual representation showcasing the comparison between a) the input trajectory and b) reconstructed counterpart using the best-performing CNN-LSTM VAE model.

Linear interpolation in the latent space is an effective approach to synthesize new data vectors, which subsequently generate diverse robot trajectories. This process involves traversing the latent dimension in a linear manner, producing a sequence of latent vectors that serve as inputs for trajectory generation. Although the interpolation occurs linearly within the latent space, the resulting output trajectories exhibit inherent non-linearities, reflecting the model's ability to capture and express complex motor neuron control patterns. The generated trajectories, derived from these interpolated latent vectors, encapsulate a wide range of learned behaviours, demonstrating the model's capacity to generalize and extrapolate diverse movement dynamics. This capability highlights the model's effectiveness in generating complex robot trajectories from seemingly linear latent representations. Fig. 6.9 illustrates this interpolation process in the latent space, highlighting the transformation of interpolated vectors into non-linear robot trajectories. Fig. 6.9a shows the canvas visualization for an overall view, while Fig. 6.9b and Fig. 6.9c present the corresponding joint space and task space trajectories, respectively, with colours indicating different instances.

Taskwise path position accuracy provides valuable insights into the model's performance across different tasks, encompassing a range of complexities and nuances. Evaluation across 10 distinct tasks reveals variability in accuracy, reflecting the model's capacity to adapt and predict path positions with varying degrees of precision. Maximum accuracy reached 4 cm, indicating high precision in certain task contexts, while minimum accuracy extends to 9.8 cm, suggesting challenges associated with specific tasks. This range highlights the model's task-specific performance and its ability to generalize across diverse scenarios. Fig. 6.10 illustrates these taskwise accuracy variations, highlighting the model's proficiency in path position prediction

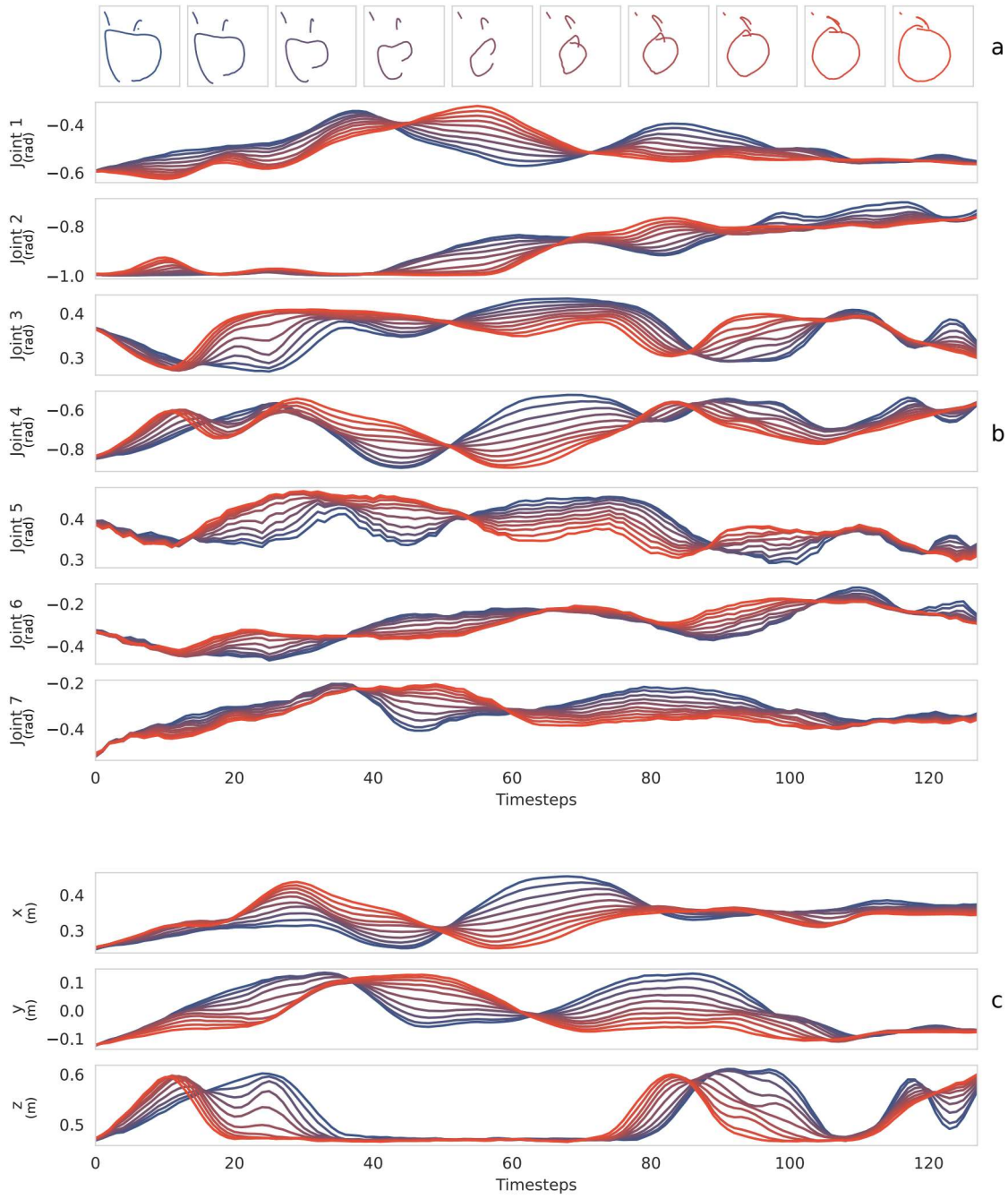


Fig. 6.9: Demonstrating the conditional one-dimensional linear interpolation of latent space for *Task1* using the best-performing CNN-LSTM VAE model. (a) Canvas visualization provides an overall view, while (b) Joint space plots and (c) Task space plots offer detailed insights, with colours representing corresponding instances.

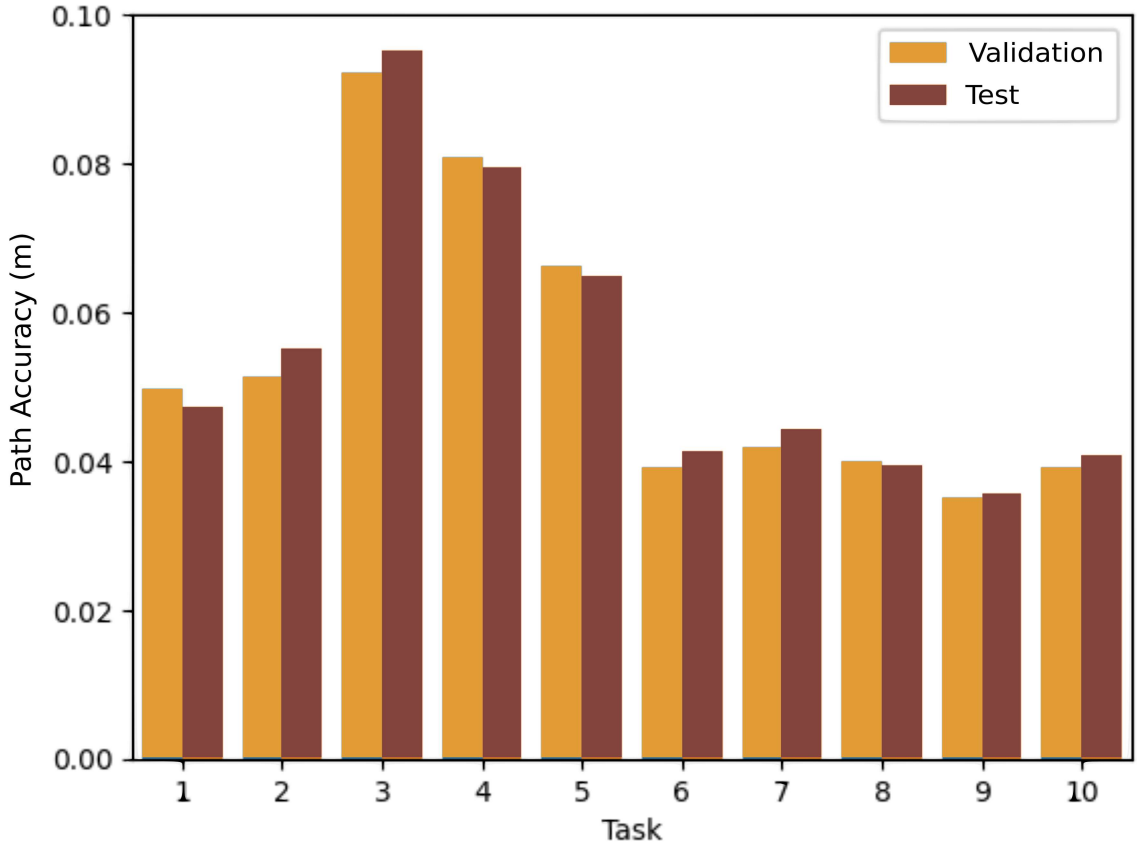


Fig. 6.10: Task-wise evaluation of path position accuracy for 10 diverse tasks using the best-performing CNN-LSTM VAE model, depicted separately for validation and test datasets. The figure provides a detailed mapping of the model's accuracy levels, aiding in task-specific performance assessment.

across multiple tasks and identifying cases that may indicate overfitting or underfitting in certain scenarios. A comparison of test and validation accuracies reveals instances where test accuracy surpasses validation accuracy, emphasizing the importance of evaluating the model's generalization on unseen data. Such instances provide insights into the model's robustness and its potential applicability beyond the training and validation environments.

A taskwise analysis of performance metrics provides detailed insights into the effectiveness of VAE models utilizing a CNN-LSTM architecture in robot trajectory planning. The analysis evaluates key metrics, including Robot Displacement Loss, Task Space Loss and Path Accuracy, across both validation and test datasets. This evaluation highlights the model's capacity to maintain consistent performance across different datasets, ensuring reliable trajectory predictions in various task scenarios.

Table 6.3 presents a detailed comparison of these performance metrics, offering a comprehensive view of the model's effectiveness in trajectory planning.

Table 6.3: Taskwise Analysis of Robot Displacement Loss, Task Space Loss and Path Accuracy in Best Performing VAE Models with CNN-LSTM: Evaluation on validation and test Datasets

Task	Robot Displacement Loss (m)		Task Space Loss (m)		Path Accuracy (m)	
	Test	Validation	Test	Validation	Test	Validation
1	0.000899	0.000958	0.000228	0.000251	0.047401	0.049772
2	0.001341	0.001275	0.000300	0.000290	0.055262	0.051318
3	0.003384	0.003382	0.000868	0.000853	0.095293	0.092171
4	0.002403	0.002383	0.000634	0.000635	0.079634	0.081003
5	0.001766	0.001732	0.000454	0.000442	0.064830	0.066155
6	0.000794	0.000744	0.000176	0.000163	0.041416	0.039131
7	0.000979	0.001053	0.000209	0.000220	0.044212	0.041799
8	0.000700	0.000734	0.000175	0.000192	0.039547	0.040125
9	0.000726	0.000825	0.000146	0.000172	0.035579	0.035208
10	0.000683	0.000604	0.000160	0.000143	0.040748	0.039264

The Robot Displacement Loss values range from $0.000683m$ to $0.003384m$ across the ten tasks. Lower values indicate more accurate predictions of robot displacement during trajectory execution. Task 10 demonstrates the lowest Robot Displacement Loss at $0.000683m$ in the test dataset. This metric quantifies the disparity between the predicted and actual displacements, serving as a critical gauge of the model's accuracy in capturing the dynamics of robot trajectories. Task 3 exhibits the highest Robot Displacement Loss values for both test ($0.003384m$) and validation ($0.003382m$) datasets, indicating a considerable challenge or intricacy in predicting the spatial movements of the robot for this specific task. Further investigation into the nature of Task 3 may reveal insights into areas for model refinement or task-specific optimisation. While some tasks, like 6 and 9, consistently demonstrate low Robot Displacement Loss values across both datasets, suggesting the models' adeptness in accurately predicting robot displacements, others, such as 3 and 5, consistently exhibit higher losses, pointing to potential complexities or challenges in these tasks.

Generally, the model demonstrates commendable performance across tasks, as indicated by the low Robot Displacement and Task Space Loss values. Path Accuracy values, while varying across the tasks, consistently emphasize the model's ability to

accurately predict trajectory paths. Tasks 6, 9 and 10 consistently exhibit noteworthy performance, achieving low displacement, minimal task space loss and competitive path accuracy. A notable observation is that, in some instances, test accuracies surpass their corresponding validation accuracies. This divergence could be attributed to variations in the complexity of the tasks and the generalization capacity of the model to unseen data.

6.4 Transformer Neural Network

Attention mechanisms were introduced to address limitations in traditional neural networks, such as Recurrent Neural Networks (RNNs) and Convolutional Neural Networks (CNNs), which often struggle with capturing long-range dependencies and handling variable-length sequences. It allows models to dynamically weigh the importance of different parts of the input, making them highly adaptive. The Transformer architecture introduced in the paper "Attention is All You Need" by Vaswani *et al.*, popularized attention mechanisms in deep learning [347]. It utilizes self-attention layers to process input sequences efficiently in parallel, making it suitable for tasks like machine translation and text generation. In recent years, the integration of Transformer-based neural network architectures into the field of robotics has yielded groundbreaking advancements in robot control, planning and perception [348, 349]. Inspired from the success in natural language processing [350] and computer vision [351], researchers have adapted Transformers to address complex challenges in robotics, making them a promising addition to the field.

The complexity of task planning in robots bears a close resemblance to that of human language, requiring the formation of subtasks, complex dependencies and coordinated collaborative tasks. This chapter provides a classic example involving the generation and motion systems of functional Transformer VAE. Inspired by transformer's groundbreaking success in understanding natural language, this chapter examines how to manipulate robots. This research covers the theoretical foundations of transformer VAEs, their integration in the robotic domain and their ability to bridge the gap between high-performance and low-level robotic actions. Additionally, it re-

flect on the profound implications and challenges of this transformational journey, moving robotics into an era of unprecedented potential and versatility.

Transformer neural networks, originally created for language processing, have proven remarkably effective in robotics [277, 278, 352]. They excel at building complex, context-aware models, which is ideal for robotic task and motion planning. This ability allows robots to adapt to their environments, optimise their actions and navigate complex real-world situations. As a result, robots using transformers are becoming more autonomous and precise in their movements. This section introduces the development of a Transformer module for robot trajectory planning, designed by incorporating a robot trajectory encoding layer that transforms data into a higher-dimensional vector. Positional encoding adds absolute position information to the representation, while the attention mechanism computes interactions between encoded data to generate the robot trajectory. The final output layer transforms the data to fit within the specified range of limits. The contribution in this section includes the development of a robot trajectory planning module by utilizing a Transformer VAE for robot trajectory data.

6.4.1 Robot Data Embedding Layer

The robot task, denoted as $\mathbf{T} = \{\mathbf{t}_1, \mathbf{t}_2, \ldots, \mathbf{t}_N\}$ with N time-steps, encapsulates the complexity of each temporal phase. At any given time-step t, the task vector $\mathbf{t}_t$ encompasses the joint positional vectors $\mathbf{q}_1, \mathbf{q}_2, \ldots, \mathbf{q}_n$, where n signifies the degree of freedom of the robot. This representation is mathematically expressed as $\mathbf{t}_t = [\mathbf{q}_1, \mathbf{q}_2, \ldots, \mathbf{q}_n]$. The Robot Data Embedding Layer processes these joint positional vectors through linear fully connected layers, employing ReLU activation. This transformative layer converts the vector of dimension equal to the degree of freedom of the robot ($\mathbb{R}^{\text{DoF}}$) into a hidden vector ($\mathbf{H} = \{\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_N\}$) of intermediate dimension(represented as d_{model}) ($\mathbb{R}^{d_{model}}$). The resulting hidden vector $\mathbf{h}_t$ encapsulates the essence of the robot's state at the given time-step. A learnable context vector embedding layer is inserted in the input of transformer encoder layer together with learnable parameters of mean and standard deviation vectors of the associated task. The implementation comprises of a fully connected neural network with ReLU non-linear activation layer as the Embedding layer.

6.4.2 Positional Encodings

Attention processes do not intrinsically capture positional information in a sequence since they are permutation-invariant. In order to rectify this, input embeddings are often enhanced using absolute positional encodings. In the context of robot trajectory data, representing time-varying joint variables, additive positional embeddings are utilized to convey crucial information about each element's location within the sequence. This ensures that the model, particularly in Transformer architectures, can discern the order and relative positions of items, enhancing its ability to capture temporal dependencies in the trajectory. The positional encoding [347, 353] $PE(pos, i)$ for a given position pos, dimensionality of input embeddings d_model and dimension i is computed as:

$$PE(pos, i) = \begin{cases} sin(pos/10000^{(2i/d_{model})}), & \text{if } i \text{ is even,} \\ cos(pos/10000^{(2i/d_{model})}), & \text{if } i \text{ is odd} \end{cases} \tag{6.9}$$

$PE(pos, i)$ yields a scalar value that is added to the corresponding dimension i of the input embedding for the element at position pos in the sequence. The proposed model uses a dimensionality equal to the latent dimension and the dataset is resampled to a sequence length of 128. The positional encoder representation of dimensionality 128 and latent dimension 32 is visualised in Fig. 6.11. The use of sine and cosine

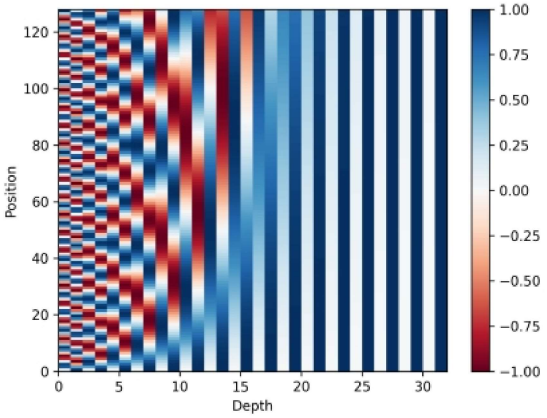


Fig. 6.11: Positional encoder used in transformer model with dimensionality of 32 and maximum position of 128

functions with varying frequencies ensures that the positional encodings are unique for each position and dimension, allowing the model to distinguish between different positions effectively.

6.4.3 Attention Mechanism

In the context of robot task planning, the attention mechanism within transformers plays a pivotal role in orchestrating complex sequences of actions and movements. Similar to how it operates in natural language processing, the attention mechanism allows robots to selectively concentrate on crucial elements within their environment and task specifications. It allows them to establish relationships between various sub-tasks, motion sequences and environmental cues, thereby capturing both short-term and long-term dependencies crucial for successful planning and execution. By dynamically allocating attention to relevant aspects of the planning process, robots can adapt to changing conditions, optimise their actions and navigate through task hierarchies. This attention-driven approach enhances the adaptability and efficiency of robots, making it a valuable tool in advancing autonomous and adaptive robotics systems.

The hidden vector $\mathbf{h}_t \in \mathbb{R}^{\text{Intermediate dimension}}$ serves as the input to the attention layer. In the attention mechanism, the query vector Q pinpoints a specific element of interest in the input sequence, guiding the allocation of attention to different elements (keys). Mathematically, Q is derived by multiplying the input sequence $\mathbf{H}$ with a learnable weight matrix W_ψ. The key vector K, representing all elements, is obtained through a distinct linear transformation using W_β, facilitating relevance computation with the query. Simultaneously, the value vector (V), associated with each element, is generated using W_v and contains crucial information for the final output. These operations collectively enable adaptive processing of the robot trajectory, dynamically focusing on pertinent elements during generation. The query, key and value vectors are mathematically calculated using equations Eq. 6.10, Eq. 6.11 and Eq. 6.12 respectively.

$$Q = \mathbf{H} * W_\psi \tag{6.10}$$

$$K = \mathbf{H} * W_\beta \tag{6.11}$$

$$V = \mathbf{H} * W_v \tag{6.12}$$

The hidden dimension serves as a higher-dimensional representation of the pose within the trajectory, encapsulating a richer and more informative characterization of the system's state. This elevated dimensional representation is crucial in generating the hidden vector within the attention mechanism, enhancing the model's capacity to discern features in the robot trajectory. In the attention process, queries, keys and values, each obtained through linear transformations of the input sequence, play distinct roles, collectively contributing to the nuanced processing of the robot trajectory and enabling the model to adaptively focus on relevant elements during generation. To calculate attention scores A, perform a dot product between the queries and keys, followed by a scaling operation:

$$A = softmax((Q * K^T)/\sqrt{d_k}) \tag{6.13}$$

Here, d_k represents the dimensionality of the keys and *softmax* activation [354] is applied element-wise to ensure that the attention scores sum to 1 for each query, making them a probability distribution over the keys. The attention scores A determine how much each element in the sequence should contribute to the output. These scores to compute a weighted sum of the values:

$$O = A * V \tag{6.14}$$

Where, O represents the output of the attention mechanism, which is a weighted sum of the values V based on the attention scores A.

Multi-head attention is a variant of attention where multiple sets of attention weights are computed in parallel. Each "head" learns different patterns and relationships in the data. The outputs from multiple heads are typically concatenated or linearly combined to produce the final attention representation as represented in Fig. 6.12. This approach enhances the model's capacity to capture diverse features. After combining the outputs from all attention heads, non-linear activation functions and feed-forward layers were applied to the concatenated output to further capture and represent complex relationships in the data.

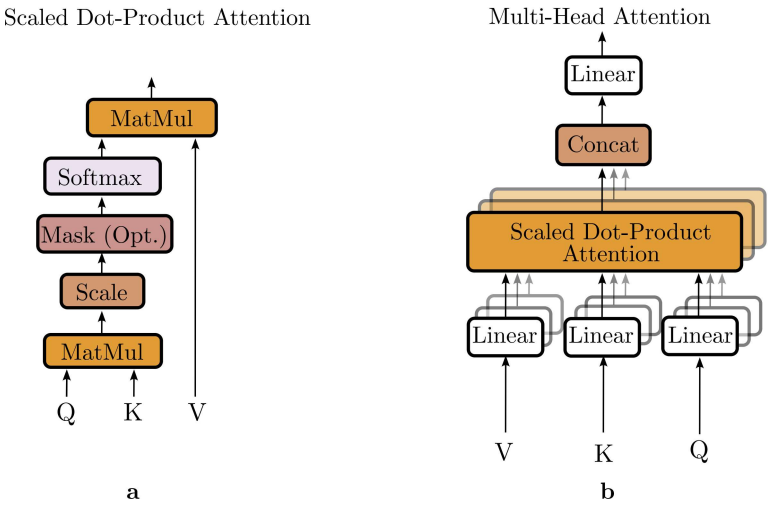


Fig. 6.12: Multi-head attention mechanism . a) Scaled dot product attention and b) the Multi-head attention block with many individual scaled dot-product attention.

6.4.4 Output Data Decoding Layer

In the output decoding layer, each hidden representation in the decoder ($\mathbf{h}_t \in \mathbb{R}^{d_model}$) serves as a vector encoding for a specific robot pose. These representations are projected back to robot joint angles ($\mathbf{t}_t = [\mathbf{q}_1, \mathbf{q}_2, \ldots, \mathbf{q}_n]$) using a linear fully connected layer. The application of a *tanh* activation function [355] in the output layer ensures that the generated joint angles are restricted within a defined limit, providing a practical and bounded output for the robot trajectory.

6.5 Attention-based Trajectory Generator Modules for Improved Performance

The proposed model is a Transformer-based VAE that processes trajectory data using stacked Transformer encoder and decoder layers. The input 1D trajectory data is embedded into a latent vector of fixed dimensionality (d_{model}), with task-specific labels embedded as mean (μ) and standard deviation (σ) vectors. These embedded inputs, combined with the μ and σ vectors, are added to positional encodings before being passed through N stacked Transformer encoder layers. Each encoder layer consists of a self-attention mechanism, followed by add and norm layers, a feedforward network and another add and norm layer. The encoder output generates the μ and σ vectors

through linear layers and the reparametrization trick technique is applied to compute the latent vector z.

The decoder initializes with a positional encoding vector, followed by self-attention and add and norm layers. The cross-attention layer processes the z vector and normalized outputs, which is further refined by add and norm layers, a feedforward network and additional add and norm layers. The decoder mirrors the encoder's structure with N stacked layers. The final output passes through a Tanh activation function, constraining the generated trajectory values between -1 and 1. Uniformity is maintained across hyperparameters such as the number of layers, activation functions, normalization locations and attention heads to optimize model convergence and efficiency.

The input trajectory $\mathbf{T}$ is concatenated with the μ and σ vectors of dimension d_{model} and the positional encodings transform each vector into the same dimensional space. The attention mechanism captures relationships between poses, aiding the model in learning sequential dependencies in robot trajectory data. The Positionwise Feedforward Network (FFN) maps each input trajectory in $\mathbb{R}^J$ to a latent vector in $\mathbb{R}^{d_{model}}$, where J denotes the degrees of freedom. Skip connections enhance gradient flow, mitigating vanishing gradient issues and facilitating the training of deeper models. The reparametrization generates μ and σ vectors in $\mathbb{R}^{d_{model}}$, forming the latent variable z. The decoder uses both positional encodings and z as memory, allowing parallel training and validation without looping the output to input. The final output layer maps $\mathbb{R}^{d_{model}}$ vectors to $\mathbb{R}^J$ vectors, completing the trajectory mapping.

The proposed generative model architecture employs Transformer layers fits into VAE architectures as illustrated in Fig. 6.13 with an abstract learnable latent vector for each of the task. The learnable Embeddings and absolute positional encoders make the proposed transformer VAE model for robot trajectory generation as illustrated in Fig. 6.13 and the architecture with configuration and number of trainable parameters for encoder and decoder are visualised in Table 6.4 and 6.5 for the model with latent dimension of 32 and ten stacked layers ($N = 10$).

The gradient-based sequence generation in Transformer VAE is facilitated through loss back-propagation, incorporating the VAE loss. For creating a more

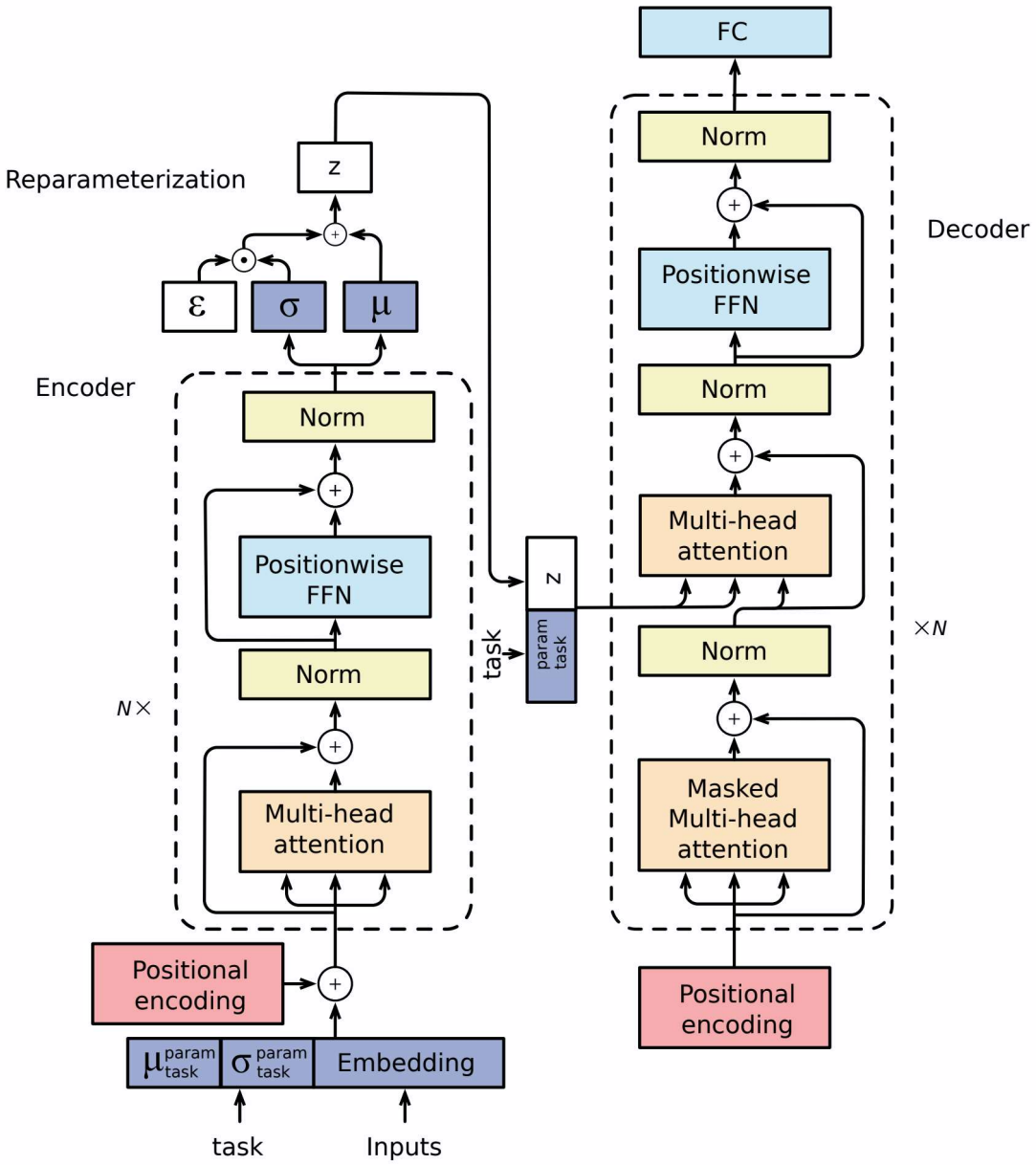


Fig. 6.13: Proposed Transformer-VAE model for offline robot trajectory learning with N stacked layers of Encoder and Decoder.

Table 6.4: Transformer encoder architecture with latent dimension of 32, 10 stacked transformer encoder layers and hidden vector dimension of 128

Layer	Configuration		Params
skelEmbedding	Linear(in_features=7, out_features=128, bias=True)		512
sequence_pos_encoder	PositionalEncoding		0
TransEncLayer(1-10)		149 K	1491 K
self_attn	MultiheadAttention(in_features=64, out_features=64, bias=True)	17 K	
linear1	Linear(in_features=64, out_features=1024)	67 K	
dropout	Dropout(p=0.1, inplace=False)	0	
linear2	Linear(in_features=1024, out_features=64)	66 K	
norm1	LayerNorm((64,), eps=1e-05)	128	
norm2	LayerNorm((64,), eps=1e-05)	128	
dropout1	Dropout(p=0.1, inplace=False)	0	
dropout2	Dropout(p=0.1, inplace=False)	0	
Total trainable parameters			1.49 M

Table 6.5: Transformer decoder architecture with latent dimension of 32 and hidden vector dimension of 128

Layer		Configuration		Params
sequence_pos_encoder		PositionalEncoding		0
TransDecLayer(1-10)			162 K	1619 K
	self_attn	MultiheadAttention(in_features=64, out_features=64, bias=True)	16 K	
	mha	MultiheadAttention(in_features=64, out_features=64, bias=True)	16 K	
	linear1	Linear(in_features=64, out_features=1024)	65 K	
	dropout	Dropout(p=0.1, inplace=False)	0	
	linear2	Linear(in_features=1024, out_features=64)	64 K	
	norm1	LayerNorm((64,), eps=1e-05)	128	
	norm2	LayerNorm((64,), eps=1e-05)	128	
	norm3	LayerNorm((64,), eps=1e-05)	128	
	dropout1	Dropout(p=0.1, inplace=False)	0	
	dropout2	Dropout(p=0.1, inplace=False)	0	
finallayer		Linear(in_features=64, out_features=7)		455
Total trainable parameters				1.65 M

simplified model, the dimension of μ, σ and embeddings are made equal to the latent dimension of VAE. Each positional encoder vector represents the same dimensional vector, make the process simple for stacking the layers. The model is trained with ADAM [325] optimiser with a *ReduceLROnPlateau* [344] learning rate scheduler, which reduces learning rate with a factor of 2, when the validation loss has stopped improving for a duration of 3 epochs. Additionally, to enhance training robustness, early stopping regularisation [340] with a patience of 5 epochs is implemented in order to avoid overfitting and resource wastage.

The VAE loss function, as defined in Eq. 3.7, consists of two primary components: the Reconstruction Loss (L_R) and the Kullback-Leibler Divergence Loss (L_{KL}). The reconstruction loss quantifies the difference between the original and reconstructed trajectories, while the L_{KL} term regularizes the latent space by minimizing the divergence between the approximate posterior and the prior distribution. This balance ensures that the learned latent representation is meaningful while maintaining the capacity to accurately regenerate the input data.

Section 6.3 introduces Eq. 6.7, which defines a refined loss function that combines the joint space loss and the robot displacement loss. This formulation accounts for the orientation of robot poses, ensuring that errors in individual joints are weighted differently based on their impact on the end-effector pose. To improve the estimation of orientation errors, the chosen rotation representation is the 6D rotation format (rot6D). Rot6D is a continuous and smooth representation of 3D rotations that avoids the singularities and discontinuities found in traditional formats such as Euler angles and quaternions.

In the rot6D format, the first two columns of a rotation matrix, each of size 3, are extracted and concatenated to form a 6D vector that uniquely defines the orientation. Given a rotation matrix $\mathbf{R} \in SO(3)$, the 6D vector is obtained as:

$$\mathbf{r}_{6D} = [\mathbf{R}_{:,1}, \mathbf{R}_{:,2}] \in \mathbb{R}^6$$

The third column of the rotation matrix can be recovered using the cross product of the first two columns to enforce orthogonality. Conversely, to reconstruct the rotation

matrix from the 6D representation, the two 3D vectors are orthogonalized using the Gram-Schmidt process to generate an orthonormal basis.

Rot6D is advantageous because it provides a smooth and continuous representation, eliminating the need to handle quaternion flips or gimbal lock. It also reduces redundancy compared to rotation matrices and ensures better gradient flow, which improves convergence in neural network-based models. When computing the Mean Squared Error (MSE) loss for orientation differences, rot6D is preferable because it transforms the problem into a standard regression task, allowing the use of Euclidean distance between the predicted and target 6D vectors. The reconstruction loss is computed as:

$$\mathcal{L}_R = \underbrace{\frac{1}{n}\sum_{i=1}^{n}(q_{0_i} - q_{1_i})^2}_{\text{joint space loss}} + \underbrace{\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{J}(f_j(q_{0_i}) - f_j(q_{1_i}))^2}_{\text{robot position loss}} + \underbrace{\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{J}\|\mathbf{r}_j(q_{0_i}) - \mathbf{r}_j(q_{1_i})\|^2}_{\text{robot orientation loss (rot6D)}} \tag{6.15}$$

where,

q_0 - Joint position vector of the original trajectory, where $q_0 \in \mathbb{R}^J$.

q_1 - Joint position vector of the reconstructed trajectory, where $q_1 \in \mathbb{R}^J$.

J - Total number of joints in the robot model.

n - Number of data samples in the batch.

f - Forward kinematics function that maps joint space positions to Cartesian space.

f_j - Forward kinematics output at the jth joint.

$\mathbf{r}_j(q_{0_i})$ - 6D rotation vector (rot6D) at the jth joint for the original trajectory, where $\mathbf{r}_j(q_{0_i}) \in \mathbb{R}^6$.

$\mathbf{r}_j(q_{1_i})$ - 6D rotation vector (rot6D) at the jth joint for the reconstructed trajectory, where $\mathbf{r}_j(q_{1_i}) \in \mathbb{R}^6$.

$\|\mathbf{r}_j(q_{0_i}) - \mathbf{r}_j(q_{1_i})\|^2$ - Mean squared error between the rot6D vectors of the original and reconstructed orientations.

The total loss function, incorporating both the reconstruction and Kullback-Leibler divergence losses, is expressed as:

$$\mathcal{L}_{VAE} = \frac{1}{n}\sum_{i=1}^{n}(q_{0_i} - q_{1_i})^2 + \frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{J}(f_j(q_{0_i}) - f_j(q_{1_i}))^2 + \frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{J}\|\mathbf{r}_j(q_{0_i}) - \mathbf{r}_j(q_{1_i})\|^2 + \beta\mathcal{L}_{KL} \quad (6.16)$$

Here, β is a weighting factor that controls the balance between the reconstruction and the regularization terms. By incorporating joint space and Cartesian space losses, this formulation ensures that both positional accuracy and task-related kinematics are accounted for in the overall model performance.

6.5.1 Performance Evaluation

The Transformer VAE for robot trajectory tasks is configured with a set of hyperparameters, as outlined in Table 6.6. The hyperparameter choices include varying the number of layers (4, 6, 8, 10), activation functions (ReLU [319], Gaussian Error Linear Unit (GELU) [356]), normalization location (pre, post) and dropout rates (0.0, 0.2). During training, the model utilizes a batch size of 32, four attention heads and a latent dimension of 32. The optimisation is performed using the ADAM optimiser with a learning rate of $1e^{-4}$ and a learning rate scheduler, specifically *ReduceLROnPlateau*, is employed to adaptively adjust the learning rate during training. These hyperparameter selections collectively contribute to the model's training and performance in capturing the intricate patterns within robot trajectory tasks.

Table 6.6: Hyperparameter under consideration for transformer model for optimisation

Hyperparameter	**Search Space**	**No. of combinations**
n_layers	4, 6, 8, 10	4
activation	ReLU , GELU	2
norm_location	Pre , Post	2
dropout	0.0, 0.2	2
Total number of combinations		32

The model training is conducted on a High Performance Computing (HPC) with an aggregate computational power of 268.4 Tera FLOating-point Operations Per

Second (TFLOPS). The comprehensive specifications and operational parameters of the HPC infrastructure utilized in this study are delineated in Appendix 7.5. A single instance of the training process consumes approximately 40 hours when executed on a single GTX 1080 GPU. With a total aggregate time requirement of 1280 hours, it would take approximately 53 days to complete. To optimise efficiency, an early termination strategy employing grid search is implemented, resulting in a substantial reduction to 345 hours, equivalent to approximately 14 days. This strategy achieves a remarkable 74% reduction in the overall training time, showcasing the effectiveness of the optimisation approach in accelerating the training process.

Hyperparameter optimization plays a crucial role in improving model performance and resource efficiency. A random forest regression model [357] is used to evaluate the importance of various parameters, highlighting that the number of layers is the most influential parameter. Increasing the number of layers enhances the model's ability to capture complex patterns, provided sufficient data is available. The dropout layer, applied during training and disabled during evaluation, improves the model's generalization by preventing overfitting. During the optimization process, non-performing runs are discarded and the best run is identified with minimal resource usage. The results of the hyperparameter optimization are depicted in Fig. 6.14, illustrating the relative importance of different parameters.

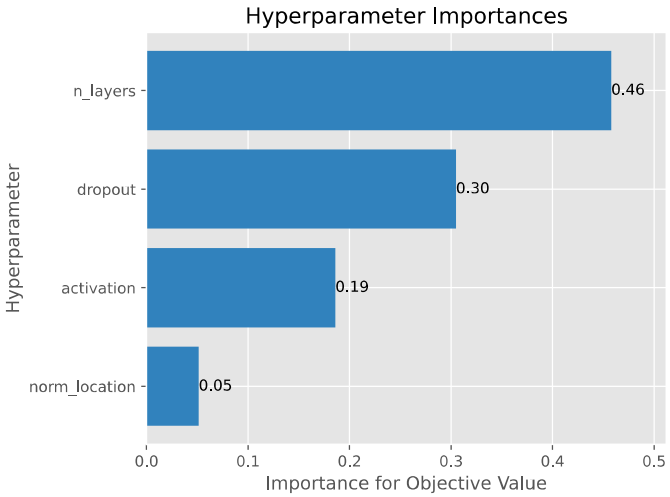


Fig. 6.14: Hyperparameter importance for the parameter tuning in transformer VAE architecture

Bayesian sampling is employed to efficiently explore the parameter space without evaluating all possible combinations. The best-performing model is selected with 10 layers, GELU activation, normalization applied after the attention layers and a dropout rate of 0.2. This model is trained for 50 epochs before being selected for deployment. Fig. 6.15 shows the evolution of validation loss across multiple trials, where most of the trials are terminated early to conserve resources. The optimization history, as shown in Fig. 6.16, demonstrates the progression towards identifying the best-performing model with minimal exploration.

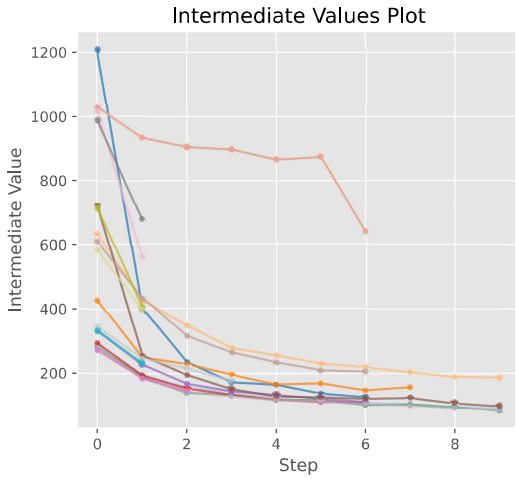


Fig. 6.15: Progression of validation loss metric of the model. The Early terminate algorithm automatically shutdown non-performing runs at the earliest.

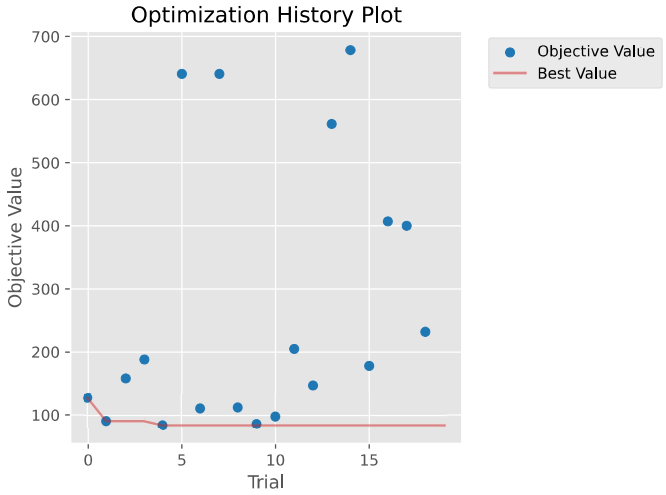


Fig. 6.16: Optimisation history of models with best model so far is represented. Validation loss is considered as the objective value for optimisation with best value highlighted in red line.

The planning and control process in deep generative models involves multiple stages that provide insights into the internal states of the model. Initially, an input task sample is used, where the motor signal serves as the input, as illustrated in Fig. 6.17a. The input is passed through the learned embedding layer, which transforms the data into a latent space suitable for further processing. The concatenated mean (μ) and standard deviation (σ) vectors are then combined with the positional encoding layer. This stage is shown in Fig. 6.17b and Fig. 6.17c, respectively. The resulting encoder vector, combined with the positional vector, contains both positional information and trajectory data. This combination enables the model to capture the sequential dependencies necessary for generating accurate trajectories.

The attention layer is responsible for capturing the correlation between different pose vectors, enabling the model to learn relationships between sequential data points. Each layer extracts unique features that contribute to embedding the information into a Gaussian distribution. The model consists of 10 layers with 4 attention heads and each latent dimensional vector has a size of 32. The independent heads process information in parallel and each row represents a different layer, while each column corresponds to an independent head. This multi-head mechanism enhances the model's ability to learn diverse representations. The visualization of the correlations captured by the attention layers across different heads and layers is shown in Fig. 6.18.

The decoder layer attention map captures the generative modelling process by progressively refining the latent vector to produce the output. As the layers advance from the latent dimensional vector to the final output, the model adds more features to the motor neural control by establishing relationships between the previous hidden vectors. This process enhances the model's capability to generate trajectories that closely resemble the original input. The attention map visualizations demonstrate that the features learned in the output layer retain significant similarities with the inherent characteristics of the original trajectory. These attention maps are depicted in Fig. 6.19.

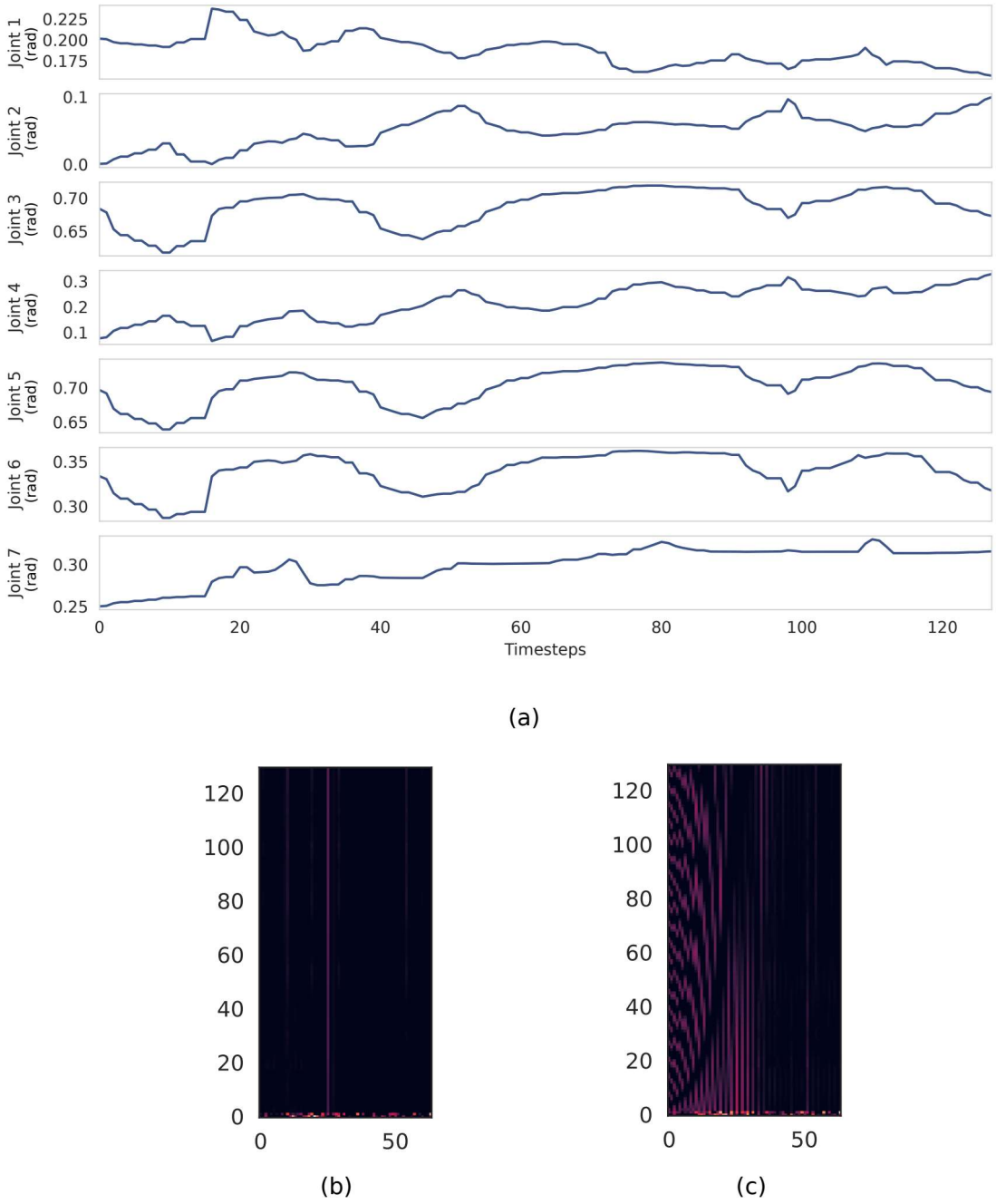


Fig. 6.17: Input trajectory encoding phases for Task 5 using the best performing Transformer VAE with Latent Dimension 32. The illustration showcases the sequential encoding stages: a) Input trajectory of an instance, b) Concatenated vector comprising μ and σ parameters along with the embedded vector and c) Resultant embedded vector after addition with positional encoding vectors.

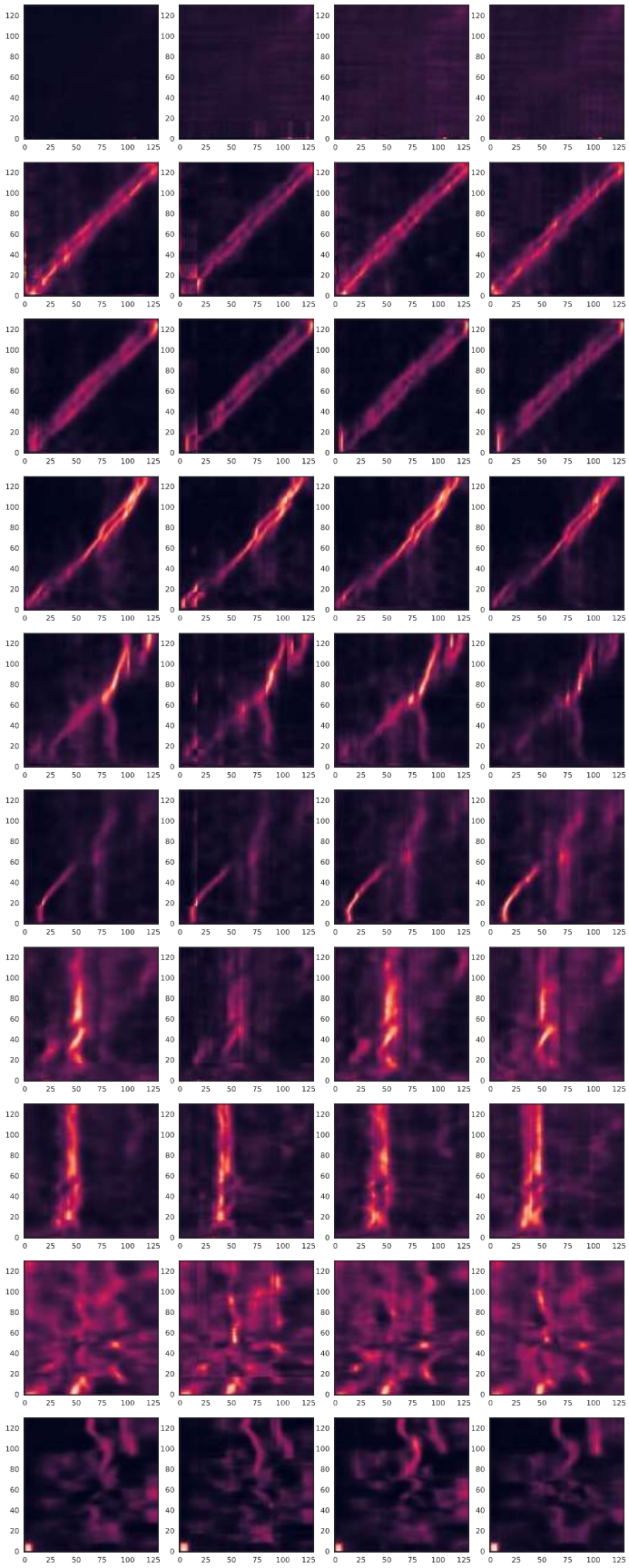


Fig. 6.18: Attention map visualisation of Transformer encoder layer in VAE Encoder with 4 heads and 10 layers.

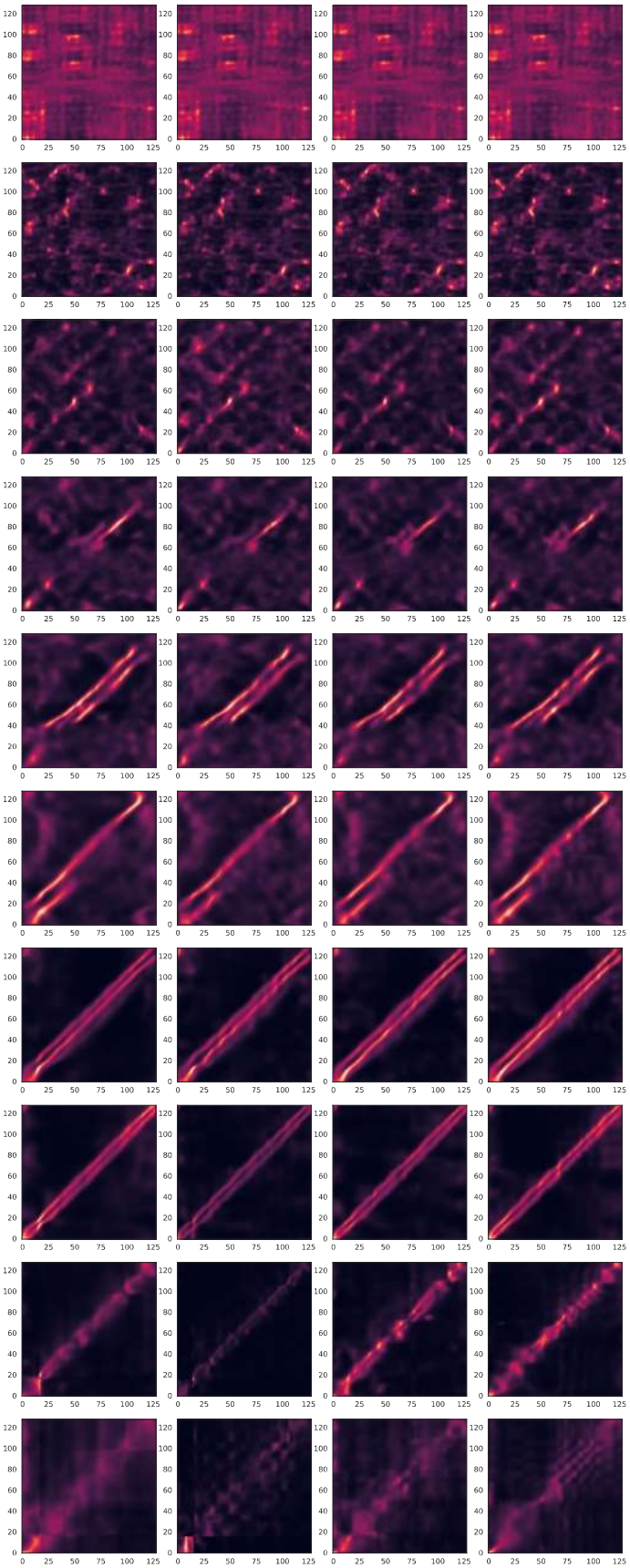


Fig. 6.19: Attention map visualisation of Transformer decoder layer in VAE Decoder with 4 heads and 10 layers.

The quantitative analysis of the Transformer VAE with varying numbers of attention heads (2, 4, 8) is presented in Table 6.7. The evaluation metrics, including Robot Displacement Loss, Task Space Loss and Path Accuracy, are reported for both test and validation datasets. Notably, the model with four attention heads outperforms others, exhibiting significantly lower losses and higher accuracy in both displacement and task space.

The model with two attention heads shows higher losses and comparatively lower accuracy, suggesting that a more intricate attention mechanism with multiple heads enhances the model's ability to capture dependencies within robot trajectories. The model with eight attention heads also performs well, but not as optimally as the four-head variant, indicating a potential trade-off between computational complexity and performance. This analysis provides valuable insights into the impact of attention head variations on the Transformer VAE's performance, aiding in the selection of an optimal configuration based on the specific requirements of the robot trajectory modelling task.

The performance of the Transformer VAE is influenced by the choice of latent dimension (d_{model}), which determines the model's capacity to capture complex dependencies in robot trajectories. Quantitative analysis is conducted with latent dimensions of 16, 32, 64 and 128 and the results are summarized in Table 6.8. The evaluation metrics include Robot Displacement Loss, Task Space Loss and Path Accuracy, reported for both test and validation datasets. As the latent dimension increases from 16 to 128, the performance metrics exhibit a consistent improvement, with models having higher latent dimensions, particularly 32, 64 and 128, demonstrating lower losses and higher accuracy in both displacement and task space. This suggests that a higher latent dimension enhances the model's ability to capture intricate features and dependencies within the trajectory data. However, beyond a certain threshold, such as from 64 to 128, the improvements become marginal. This indicates a trade-off between computational complexity and model performance, where selecting an optimal latent dimension requires balancing task requirements with resource constraints. The results

Table 6.7: Performance evaluation of Transformer-VAE with various number of heads.

No. Heads	Robot Displacement Loss (m)		Task Space Loss (m)		Path Accuracy (m)	
	Test	Validation	Test	Validation	Test	Validation
2	0.008992	0.009584	0.002281	0.002512	0.147401	0.149772
4	0.001341	0.001275	0.000300	0.000290	0.055262	0.051318
8	0.003384	0.003382	0.000868	0.000853	0.095293	0.092171

Table 6.8: Performance evaluation of Transformer-VAE with various latent dimension with 10 layer Transformer VAE architecture

Latent dimension	Robot Displacement Loss (m)		Task Space Loss (m)		Path Accuracy (m)	
	Test	Validation	Test	Validation	Test	Validation
16	0.008998	0.009584	0.002282	0.002513	0.127401	0.139772
32	0.001341	0.001275	0.000300	0.000290	0.055262	0.051318
64	0.001284	0.001282	0.000268	0.000253	0.048293	0.048171
128	0.001275	0.001275	0.000258	0.000248	0.048176	0.048274

provide insights into the effect of varying latent dimensions on the effectiveness of the Transformer VAE in modelling robot trajectories.

6.6 Modified Transformer-VAE model

The Transformer VAE maintains the same size for the latent space vector and intermediate vector dimension (d_{model}), which limits the model's performance. To address this limitation, the architecture is modified by increasing d_{model} and integrating projection layers at the robot data embedding and output layers. This adjustment enhances the model's ability to handle more complex trajectory data while maintaining manageable computational complexity. The addition of projection layers allows the latent dimension to be computed in a lower-dimensional space while performing intermediate attention computations in a higher-dimensional space. This approach improves the model's flexibility and capacity without requiring an excessive number of layers. The modified architecture of the network is illustrated in Fig. 6.20, showcasing the Modified Transformer VAE design where the model dimension exceeds the latent space dimension to enable richer feature encoding. This structural enhancement is intended to improve the expressiveness and generalization of the latent representations. Detailed network configurations corresponding to this architecture are provided in Table 6.9.

Further improvements are achieved by utilizing a Transformer encoder within the VAE decoder, enabling the model to simultaneously learn from the latent dimension and proprioception feedback. This dual-source learning mechanism enhances the model's capability to capture intricate dependencies within trajectory data. Additionally, the modified model processes fixed-length trajectory data at the output, eliminating the need for an additional attention layer at the final stage. This architectural refinement reduces computational overhead while improving the model's overall performance.

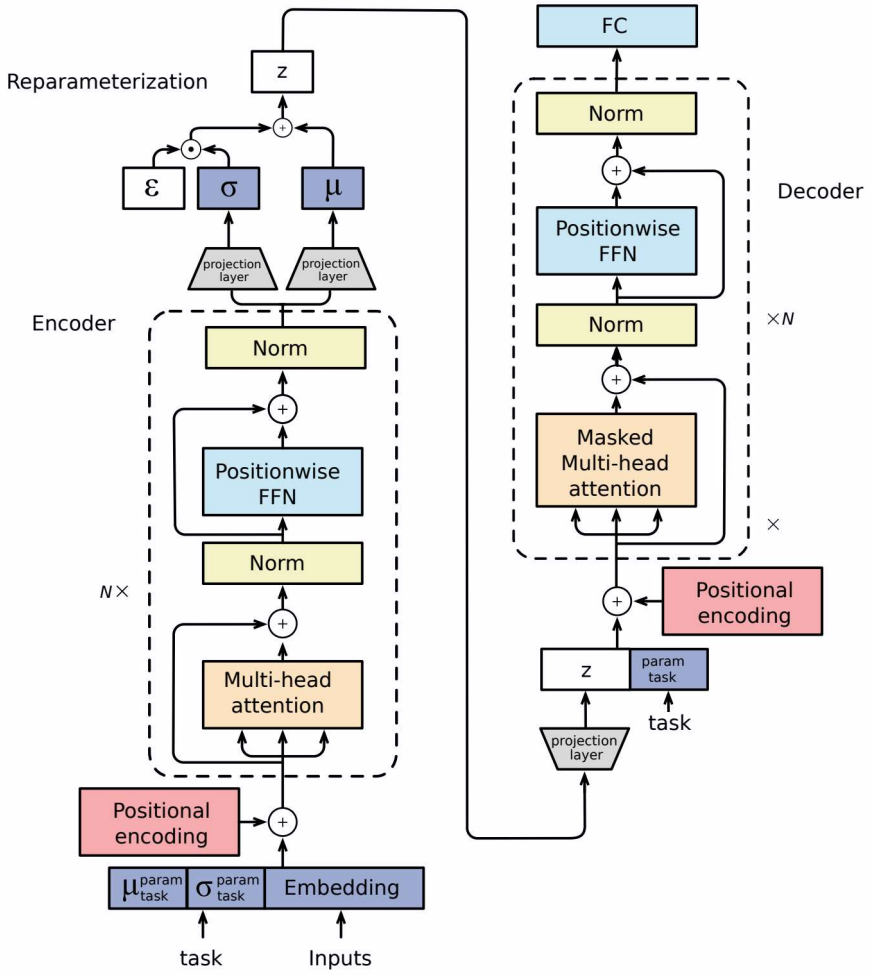


Fig. 6.20: Enhanced Modified Transformer VAE Architecture with d_{model} exceeding *latent_dimension*. The diagram highlights the augmented structural complexity, emphasizing the increased dimensionality for richer feature representation in the latent space.

Table 6.9: Transformer encoder architecture with latent dimension of 32 and hidden vector dimension of 128

	Layer		Configuration	Params	
Encoder	skelEmbedding		Linear(in_features=7, out_features=128, bias=True)		1.0 K
	sequence_pos_encoder		PositionalEncoding		0
	TransEncLayer(1-10)			329 K	10×329 K
		self_attn	MultiheadAttention(in_features=128, out_features=128, bias=True)	66 K	
		linear1	Linear(in_features=128, out_features=1024)	132 K	
		dropout	Dropout(p=0.1, inplace=False)	0	
		linear2	Linear(in_features=128, out_features=1024)	132 K	
		norm1	LayerNorm((128,), eps=1e-05)	256	
		norm2	LayerNorm((128,), eps=1e-05)	256	
		dropout1	Dropout(p=0.1, inplace=False)	0	
		dropout2	Dropout(p=0.1, inplace=False)	0	
	muenc		Linear(in_features=128, out_features=32)		4.1 K
	sigmaenc		Linear(in_features=128, out_features=32)		4.1 K
	Total trainable parameters				3.3 M
Decoder	zdec		Linear(in_features=32, out_features=128)		4.2 K
	embeddingLayer				2.1 M
	sequence_pos_encoder		PositionalEncoding		0
	TransDecLayer(1-10)			329 K	$10 \times 329K$
		self_attn	MultiheadAttention(in_features=128, out_features=128, bias=True)	66 K	
		linear1	Linear(in_features=128, out_features=1024)	132 K	
		dropout	Dropout(p=0.1, inplace=False)	0	
		linear2	Linear(in_features=128, out_features=1024)	132 K	
		norm1	LayerNorm((128,), eps=1e-05)	256	
		norm2	LayerNorm((128,), eps=1e-05)	256	
		dropout1	Dropout(p=0.1, inplace=False)	0	
		dropout2	Dropout(p=0.1, inplace=False)	0	
	finallayer		Linear(in_features=128, out_features=7)		903
	Total trainable parameters				5.4 M

6.6.1 Performance Evaluation

In order to reduce further computations, the hyperparameters obtained from the previous transformer model is utilised for the new model. The model is evaluated with various hidden latent vectors (d_{model}) of 32, 64, 128, 256 and 512 and the results are displayed in Table 6.10 with a VAE latent vector of dimension 32. It is evident that the model performs better as the hidden vector size increases, but the performance degradation happens when it is 512, which is a large model of 75M learnable parameters, it may be due to the insufficient amount of data for the model.

The Transformer VAE model for robot trajectory data $\mathbf{T}$ adeptly learns the integrated planning and control, forming a neural model for trajectory generation. A normalized input trajectory, exemplified in Fig. 6.21a, undergoes a transformative process through layers of embeddings and positional encodings, yielding the hidden representation illustrated in Fig. 6.21b-6.21d. An in-depth analysis of learned attention maps, including encoder attention patterns in Fig. 6.22 and decoder attention in Fig. 6.23, unveils the model's capacity to discern and prioritize specific segments of the input trajectory throughout encoding and decoding. Visualizing attention maps provides valuable insights into crucial regions and focal points within the trajectory data. The encoder attention maps reveal how the model strategically attends to different elements of the input trajectory when forming the latent representation, while decoder attention maps showcase the model's utilization of information from the latent space and input trajectory to generate the reconstructed output trajectory. This thorough examination aids in understanding the model's attentional mechanisms, shedding light on which parts of the input trajectory significantly contribute to the generation of the output trajectory. Such visual interpretation is pivotal for validating the model's ability to capture relevant features and dependencies within robot trajectory data. Moreover, a visual inspection affirms that the input trajectories in Fig. 6.21a and output trajectories in Fig. 6.24 share similar distributions, emphasising the model's proficiency in achieving coordinated control and effectively learning the dynamics of robot trajectories.

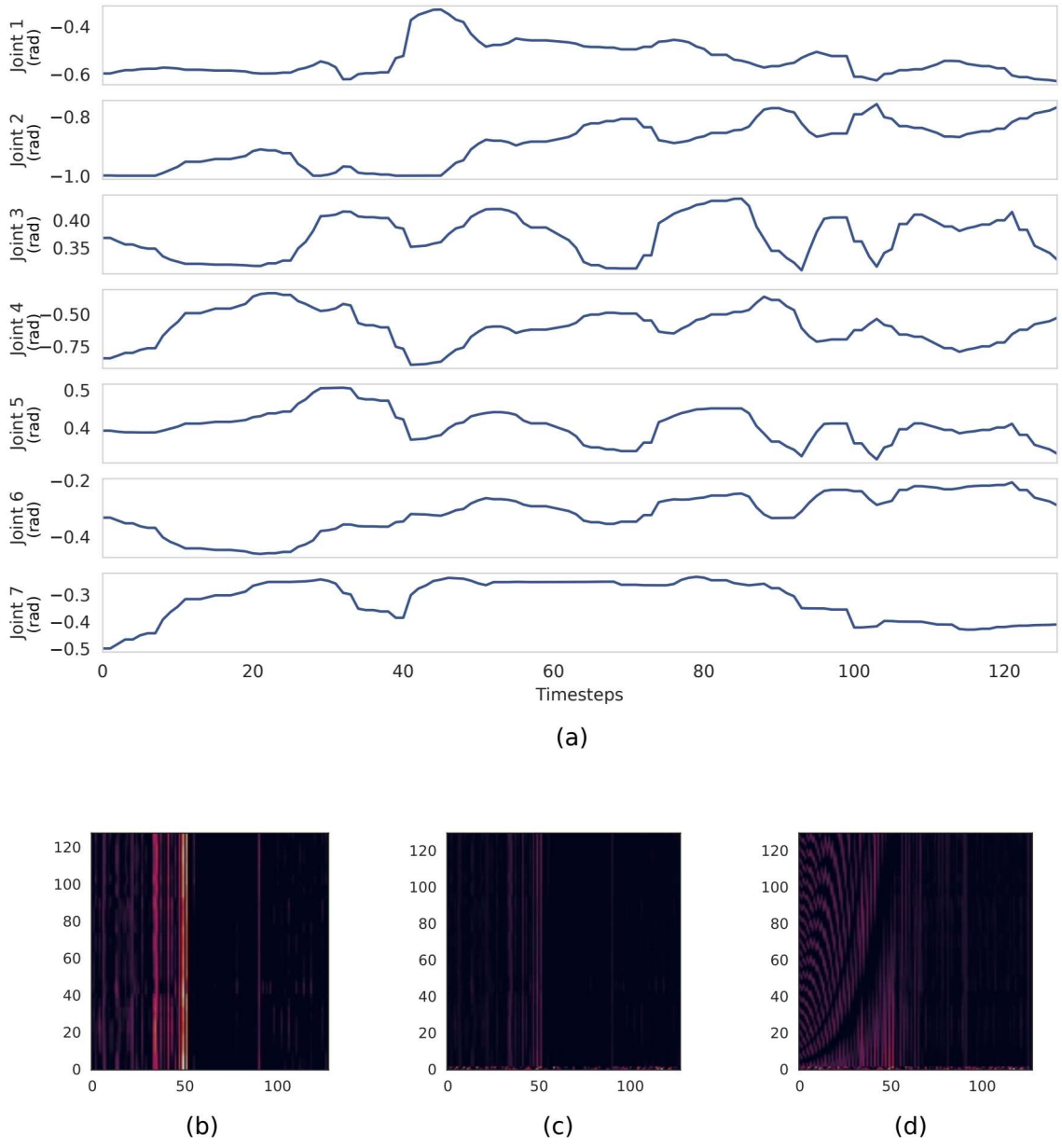


Fig. 6.21: Input data encoding phases. a) Input sample of trajectory of an instance b) Embedded vector of the input represented as input c) Concat vector of μ and σ parameters along with the embedded vector d) resultant embedded vector added with positional encoding vectors

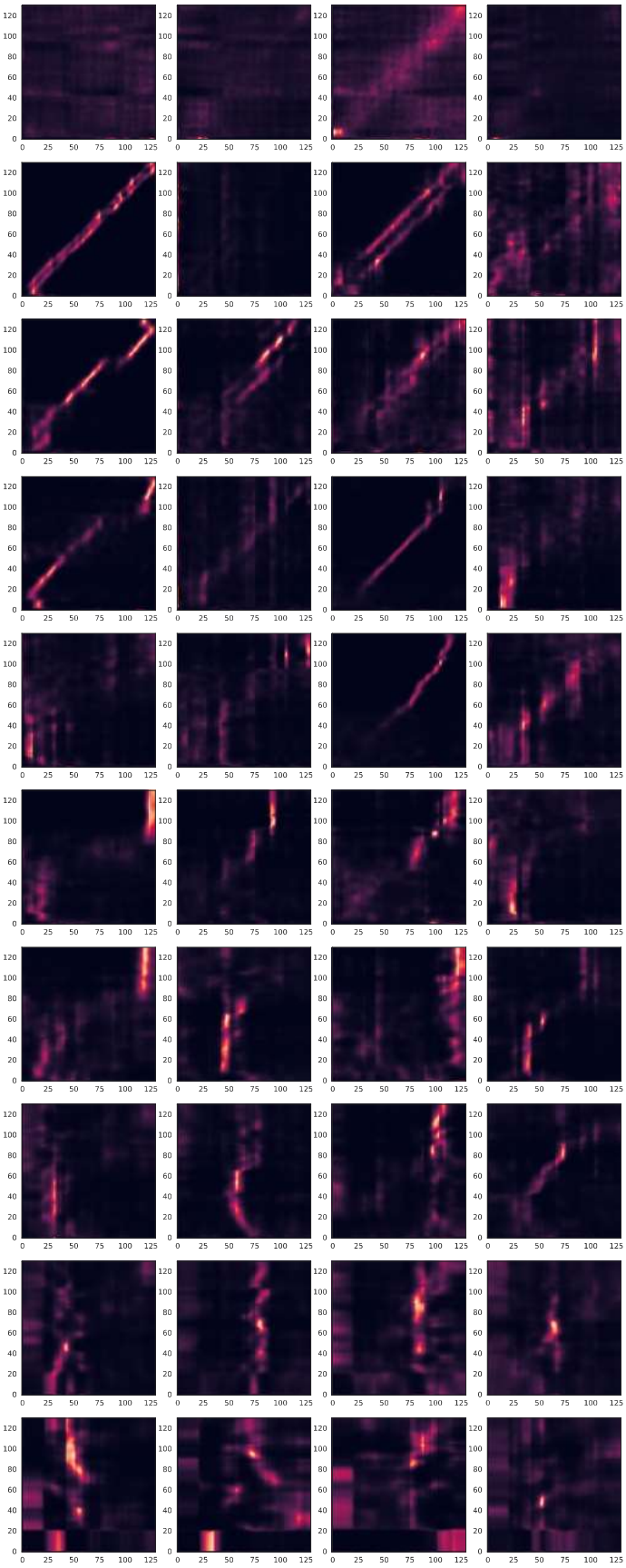


Fig. 6.22: Attention map visualisation of Transformer encoder layer in VAE Encoder with 4 heads and 10 layers.

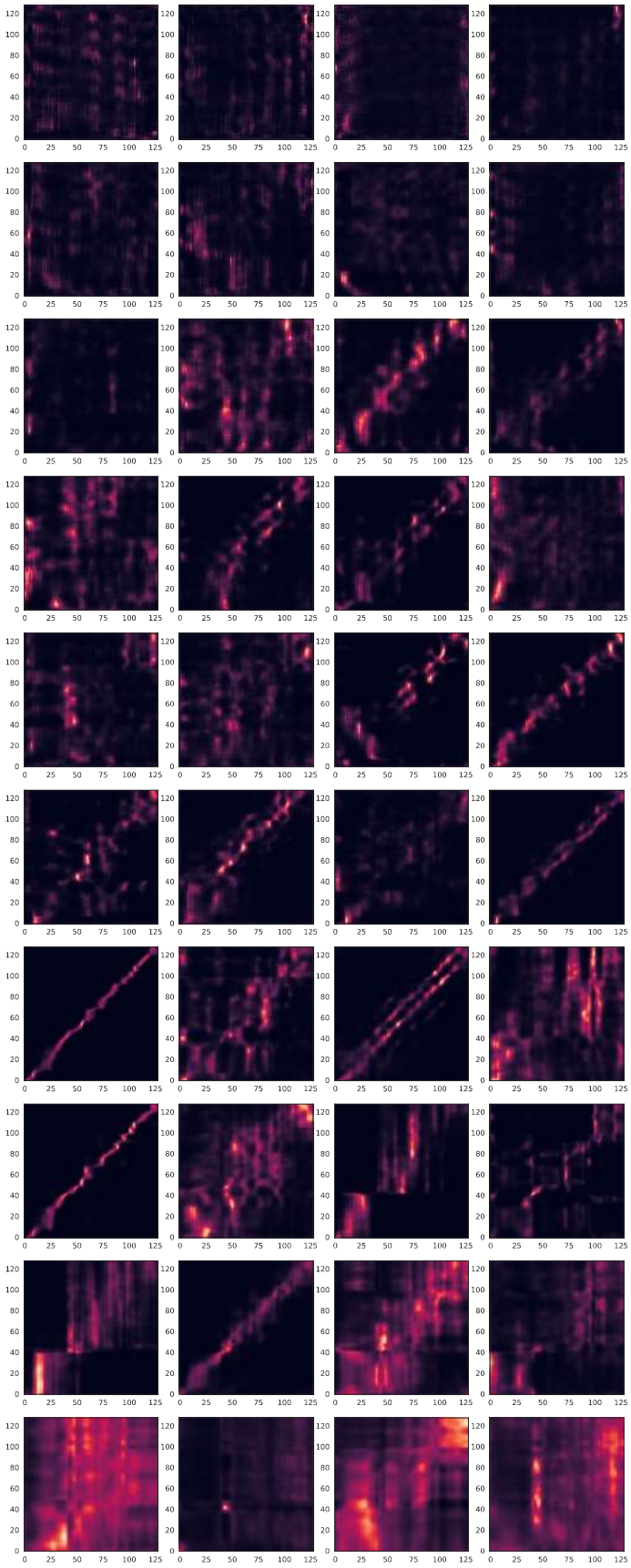


Fig. 6.23: Attention map visualisation of Transformer decoder layer in VAE Decoder with 4 heads and 10 layers.

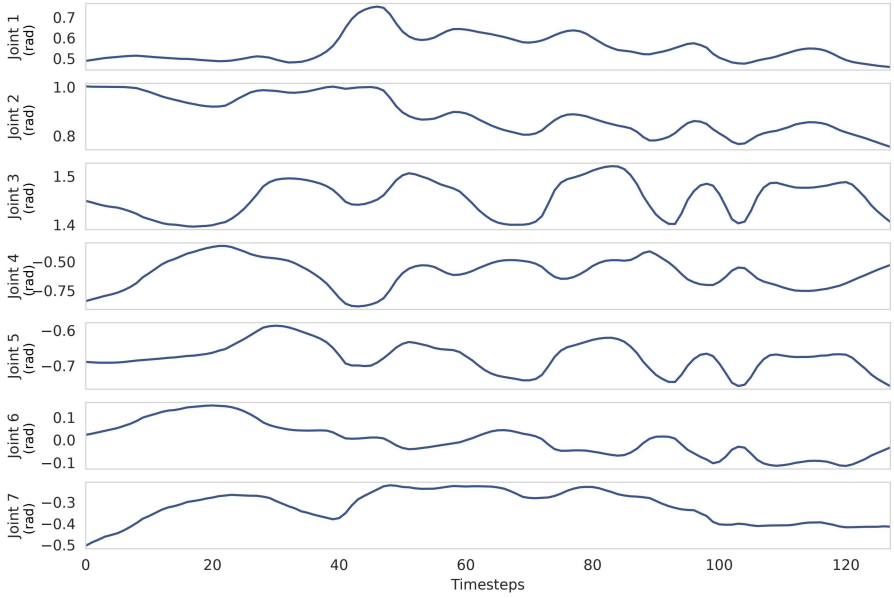


Fig. 6.24: Reconstructed output of the input trajectory in Fig. 6.21a.

The effectiveness of the model is assessed by comparing the input trajectories with the reconstructed projections. The reconstruction process evaluates the model's ability to accurately replicate the original trajectories in both joint and task spaces. This comparison serves as a quantitative metric to analyse the model's proficiency in capturing and representing the dynamics of robot movements. The reconstructed trajectories closely align with the input trajectories, demonstrating the model's capability to maintain the structural integrity of the original data. Fig. 6.25 illustrates the visual representation of the input (Fig. 6.25a) and reconstructed projections (Fig. 6.25b), These visualizations illustrate the model's effectiveness in learning and reconstructing the underlying dynamics of the input trajectories.

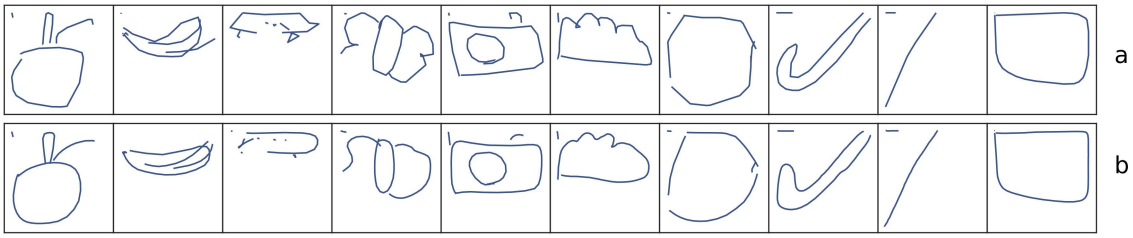


Fig. 6.25: The projected view of task space visualisation of a) input and b) reconstruction of trajectories with modified transformer VAE model with latent dimension of 32 and hidden dimension of 128.

Linear interpolation is an effective method for generating new trajectories by utilizing existing or randomly generated trajectories. This approach enables smooth transitions between key points, producing coherent and realistic trajectories in both joint and task spaces. The interpolated trajectories provide insights into the model's ability to capture complex, non-linear patterns inherent in robot motion. The joint space plots depict the articulation of the robot's joints during trajectory execution, reflecting the model's capacity to generate dynamic and realistic movements. Simultaneously, task space plots illustrate the end-effector's trajectory in the operational space, offering a comprehensive view of the robot's spatial manoeuvring. The interpolation highlights the model's capability to seamlessly interpolate between key points, capturing non-trivial dependencies in both joint and task spaces. The visual representation of the interpolated trajectories and corresponding plots in joint and task spaces is shown in Fig. 6.26, demonstrating the effectiveness of this evaluation method. Fig. 6.26a illustrates the projected view of the interpolated task *Task4* on a canvas. The joint space representation in Fig. 6.26b and corresponding task space trajectory in Fig. 6.26c. These visualizations demonstrate the effectiveness of the evaluation method using conditional one-dimensional linear interpolation.

The performance evaluation of the modified Transformer VAE with various hidden dimensions (64, 128, 256) and a fixed latent dimension of 32 is presented in Table 6.10, alongside results from the previous Transformer VAE implementation with a hidden dimension of 32. In terms of Robot Displacement Loss, Task Space Loss and Path Accuracy, the model with a hidden dimension of 128 demonstrates superior performance on both test and validation datasets. Notably, the model with a hidden dimension of 256 exhibits a significant increase in Robot Displacement Loss and Task Space Loss, indicating a potential overfitting issue. These results suggest that while increasing hidden dimension may enhance certain aspects of performance, careful consideration is required to avoid compromising overall model effectiveness. The fixed latent dimension of 32 across all models ensures a fair comparison, emphasizing the impact of hidden dimension variation. Overall, this quantitative analysis

provides valuable insights into the nuanced trade-offs associated with different hidden dimensions in the context of robot trajectory tasks.

The quantitative analysis of the modified Transformer VAE with various reconstruction loss functions is presented in Table 6.11. The three considered loss

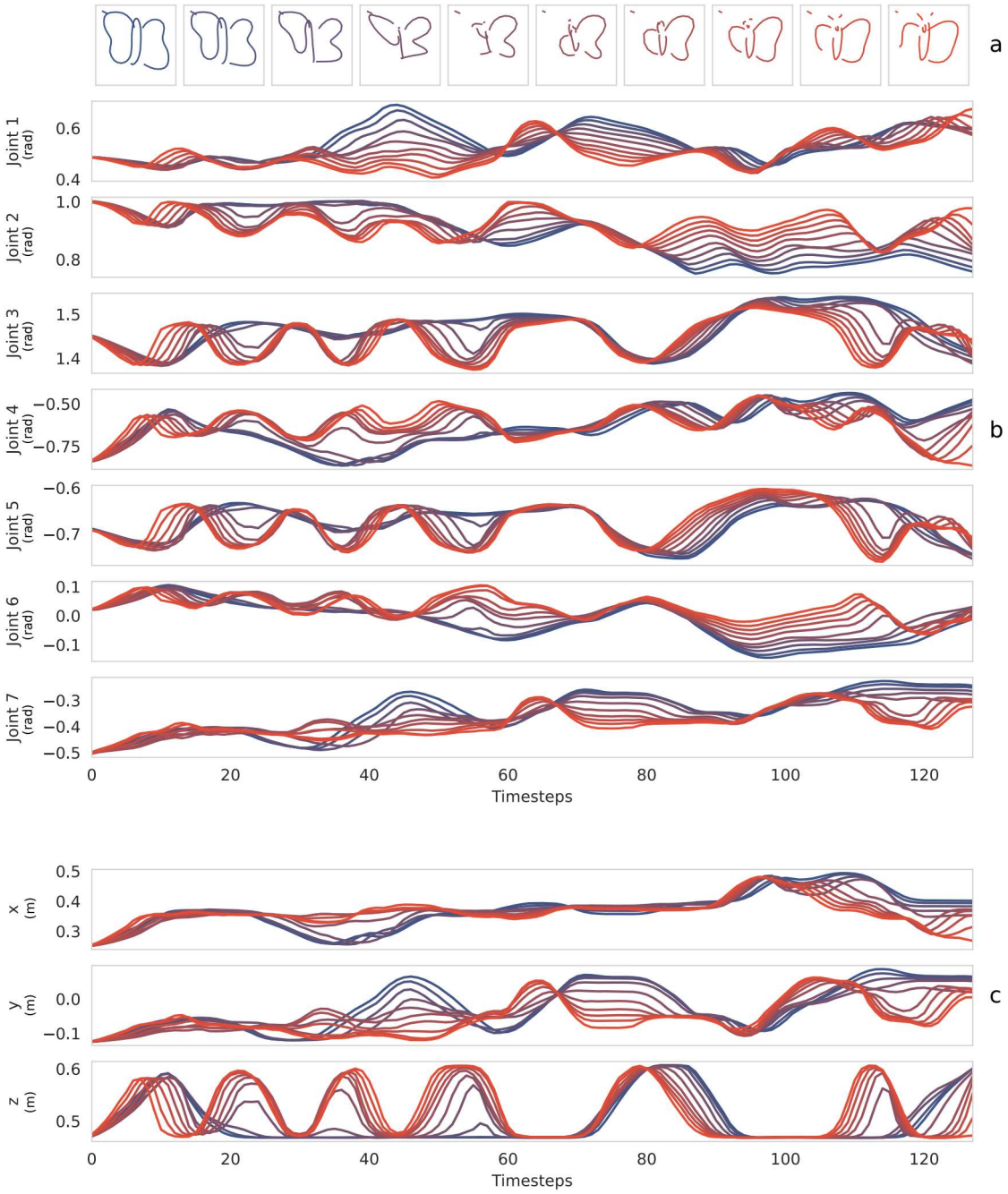


Fig. 6.26: Conditional one dimensional linear interpolation of z space for *Task4* with joint space and task space path visualised. a) The projected view of the interpolated task on a canvas b) The joint space representation and c) End-effector task space cartesian trajectory.

Table 6.10: Performance evaluation of Transformer-VAE with various hidden dimension and latent dimension of 32. 32* represents the transformer model discussed in section 6.5 for comparison

Hidden Dimension	Robot Displacement Loss (m)		Task Space Loss (m)		Path Position Accuracy (m)	
	Test	Validation	Test	Validation	Test	Validation
32*	0.001341	0.001275	0.000300	0.000290	0.055262	0.051318
64	0.000930	0.000990	0.000247	0.000244	0.047577	0.047276
128	0.000753	0.000734	0.000196	0.000189	0.044699	0.043023
256	0.065389	0.056834	0.007111	0.006991	0.165180	0.164811

Table 6.11: Performance Evaluation of the Modified Transformer VAE with various reconstruction loss functions and a latent dimension of 32. The table presents results for different loss functions, including 'rc' (joint reconstruction loss only), 'rc+fkpos' (joint reconstruction loss plus displacement of all robot link positions in the task space) and 'rc+fkpos+fkrot' (along with 'rc+fkpos' loss, considering the displacement of all robot link orientations)

Loss Function	Robot Displacement Loss		Task Space Loss		Path Accuracy	
	Test	Validation	Test	Validation	Test	Validation
rc	0.00187	0.00195	0.00050	0.00051	0.06283	0.06372
rc+fkpos	0.00075	0.00073	0.00020	0.00019	0.04470	0.04302
rc+fkpos+fkrot	0.00074	0.00077	0.00019	0.00019	0.04358	0.04357

functions include 'rc,' where only joint reconstruction loss is considered (Eq. 5.3); 'rc+fkpos,' which adds the displacement of all robot link positions in the task space to the joint reconstruction loss (Eq. 6.8); and 'rc+fkpos+fkrot,' which further includes the displacement of all robot link orientations in the task space (Eq. 6.16). The evaluation metrics encompass Robot Displacement Loss, Task Space Loss and Path Accuracy for both test and validation datasets. Notably, the 'rc+fkpos' loss function performs favorably across all metrics, showcasing lower losses and improved accuracy compared to other variants. The inclusion of both position and orientation information in 'rc+fkpos+fkrot' contributes to refining the model's performance, yielding competitive results. This quantitative assessment provides valuable insights into the effectiveness of different reconstruction loss functions in capturing and optimising the nuances of robot trajectory data, guiding the selection of an appropriate loss function based on the specific requirements of the task.

The performance of the modified Transformer VAE is further evaluated through a task-wise analysis of Path Accuracy, which provides insights into the model's adaptability across various trajectory tasks. The analysis is conducted on the best-performing model with a latent dimension of 32 and a hidden dimension (d_{model}) of 128. Path Accuracy values range from less than 8 cm for the least effective cases to approximately 2 cm for the most accurate models. This range highlights the influence of factors such as trajectory diversity, model complexity and the inherent complexity of individual tasks on the overall performance.

The task-wise variations in Path Accuracy illustrate the model's capability to adapt and generalize across different task scenarios, emphasizing the importance of considering task-specific factors during performance evaluation. The observed results provide a comprehensive understanding of the model's effectiveness in maintaining high accuracy across diverse trajectory data. The visual representation of the Path Accuracy values for various tasks is presented in Fig. 6.27.

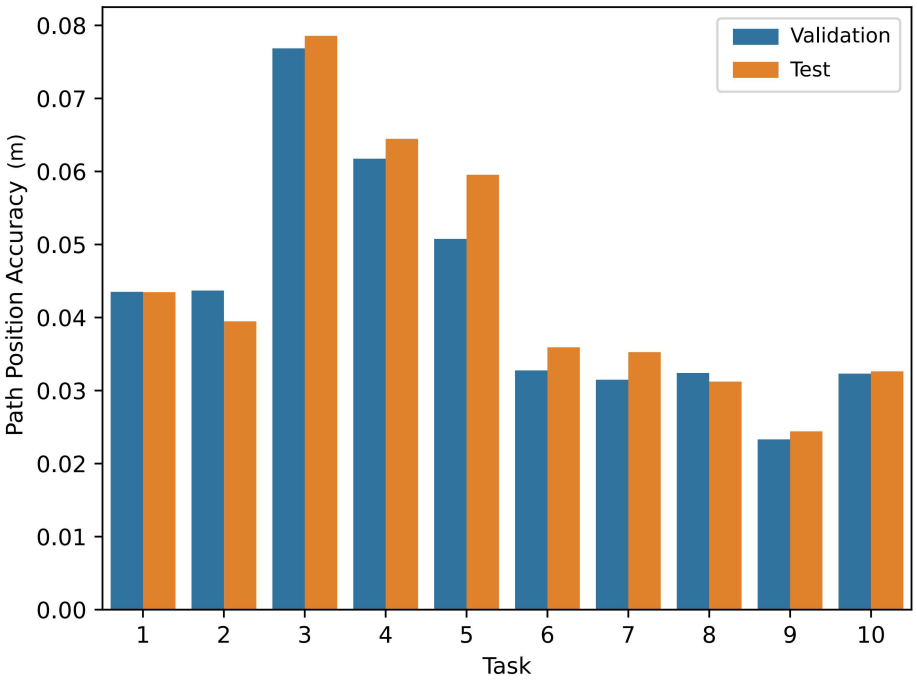


Fig. 6.27: Taskwise Robot Path Accuracy of the Modified Transformer VAE Model with a Latent Dimension of 32 and Hidden Vector Dimension (d_{model}) of 128. The plot showcases the performance across various tasks, measured in terms of path accuracy, evaluated on both validation and test datasets.

6.7 Summary

The integration of generative artificial intelligence models, including Variational Autoencoder, Long Short-Term Memory and Convolutional Neural Network, has proven effective in enhancing the precision of robot trajectory generation. The combined path accuracy, quantified at approximately 5.5 cm, highlights the model's ability to capture latent features, temporal dependencies and spatial patterns within trajectory data. The VAE component captures latent representations, enabling diverse and realistic trajectory generation, while the sequential nature of LSTM layers models temporal dependencies, ensuring smooth and coherent outputs. Simultaneously, CNN layers enhance the model's ability to extract spatial features, allowing the model to better understand complex patterns.

The exploration of a Transformer VAE and its modified version further strengthens the trajectory modelling capabilities by incorporating attention mechanisms that

capture sequential dependencies effectively. The models demonstrate their capability to generate diverse trajectory patterns while maintaining high accuracy in path planning. The modified Transformer VAE, with additional projection layers and increased hidden dimensions, improves trajectory generation, achieving a path accuracy of approximately 4.4 cm. Task-wise evaluation of path accuracy, interpolation techniques and trajectory reconstruction collectively provide a thorough assessment of the models' effectiveness in capturing and reproducing robot trajectory dynamics. The results underscore the potential of attention-based models in enhancing trajectory generation tasks and advancing generative AI techniques for robotic motion planning and control.